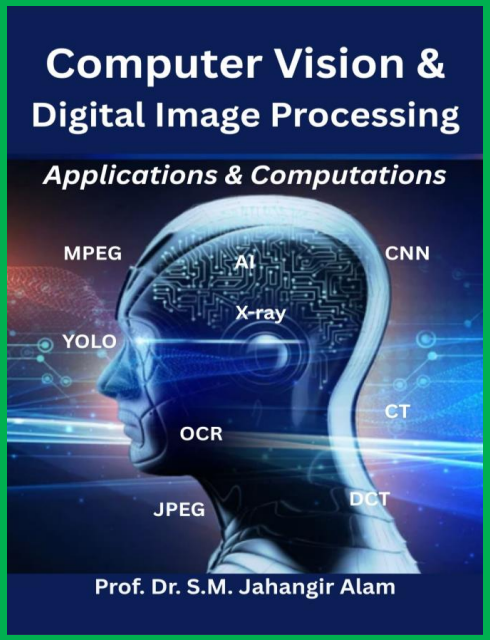
Lecture 7.1

Image Compression



Instructor Name:

Prof. Dr. Jahangir Alam

Dept. of CSE, BAUST

Image Compression

- Image compression is the process of reducing the size (in bits) of a digital image without significant loss of quality.
- Reduce the number of bits required to store/transmit an image **without significant loss** of information.

- It aims to remove redundant and irrelevant information, so that the compressed image requires less storage and less transmission time.

Image Compression Fundamentals

DVD: Digital Video Disk

Let, 2 h standard definition (SD) TV movie using $720 \times 480 \times 24-$ bit pixel arrays. A digital movie or video is a sequence of video frames in which each frame is a full-color still image. Because video players must display the frames sequentially at rates near 30 fps, SD digital video data must be accessed at,

$$30\ fps \times (720 \times 480)\ pixels\ per\ frame\ (ppf) \times 3\ bytes\ per\ pixel\ (Bpp) = 31{,}104{,}000\ bytes\ per\ second\ (Bps)$$

And, two-hour movie consists of,

$$31{,}104{,}000\ Bps \times (60 \times 60)\ seconds\ per\ hrs \times 2\ hrs \cong 2.24 \times 10^{11}\ bytes = 224\ \text{GB}$$

Image Compression Fundamentals

- ✓ 8.5 GB dual-layer conventional 12 cm 27 disks are needed to store it. To put a 2 hrs movie on a single DVD, each frame must be compressed-on average-by a factor of 26.3.

- ✓ The compression must be even higher for HDTV, where image resolutions reach $1920 \times 1080 \times 24 - \text{bits/image}$.

Why Image Compression is Needed

- ✓ Digital images contain huge amounts of data
- ✓ Example: A 512×512 grayscale image (8 bits/pixel)

$$512 \times 512 \times 8 = 2{,}097{,}152\ bits = 256\ KB$$

For color (RGB), it becomes $3 \times 256\ KB = 768\ KB$ — and much more for HD or 4K images.

Fundamentals

Compression is crucial for:

- ✓ Reducing storage space
- ✓ Reducing transmission bandwidth
- ✓ Speeding up image sharing and streaming
- ✓ Efficient database management

Bit Rate:

$$R = \frac{N_2}{M \times N}$$

$$= \frac{L}{N}\ bits/pixel\ (bpp)$$

Relative data redundancy: $R = 1 - \frac{1}{C}$

Relative Data Redundancy:

Compression ration, C: $C = \frac{b}{b'}$

If C=10, The ratio is 10:1; then R=0.9 (90%)

The Compression Ratio (CR):

$$CR = \frac{Original\ Size}{Compressed\ Size} = \frac{N_1}{N_2}$$

Compression Efficiency:

$$\eta = (1 - \frac{1}{CR}) \times 100\%$$

Fundamentals

- Error or Distortion Model:

$$MSE = \frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}\left[f(x,y) - \left(f - \hat{f}\right)\right]^2$$

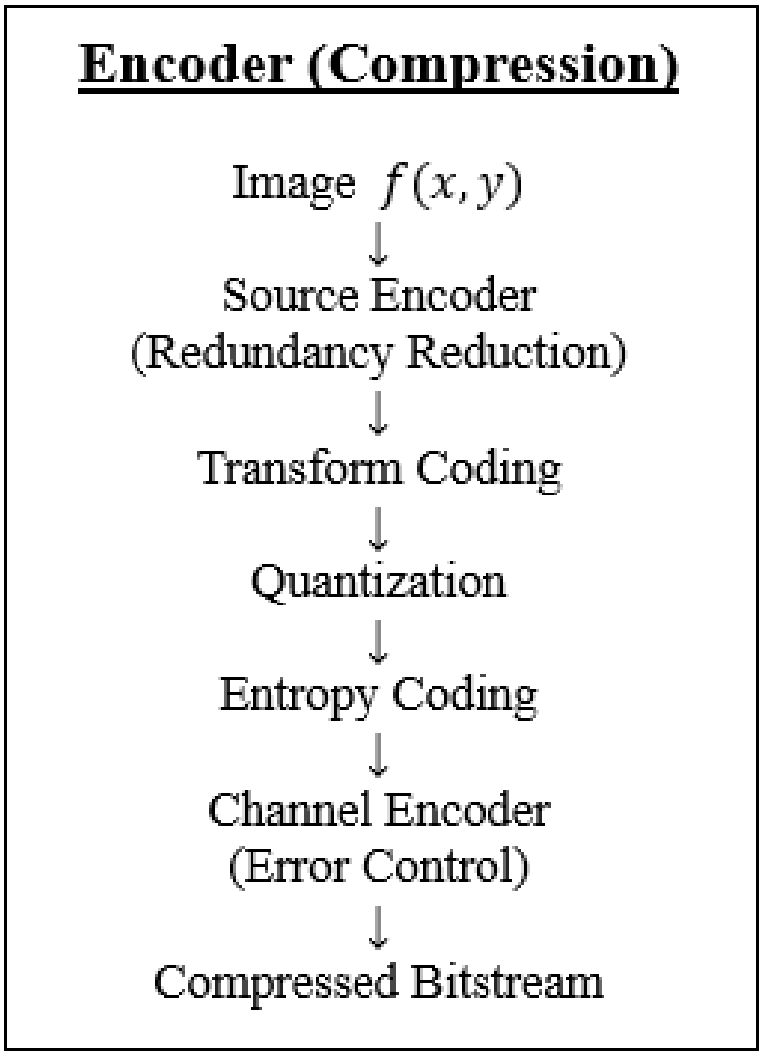
where $\hat{f}(x,y)$ is the reconstructed image.

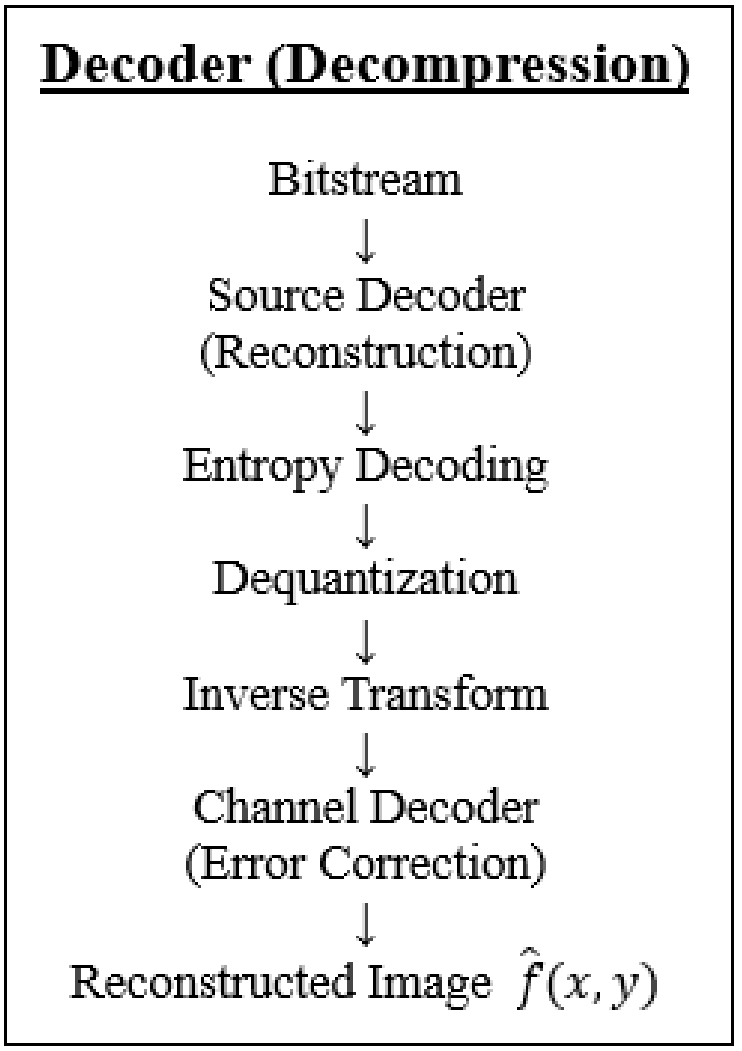
- **PSNR (Peak Signal-to-Noise Ratio):**

$$PSNR = 10log_{10}\left(\frac{255^2}{MSE}\right)\ dB$$

- The higher PSNR, the better reconstruction quality.

Basic Compression Model





Types/Classification of Image Compression:

A. Lossless Compression

- ✓ The decompressed image => original image
- ✓ Used in medical, satellite, scientific, or legal applications where accuracy is critical.

Common Techniques:

- ✓ Run-Length Encoding (RLE)
- ✓ Huffman Coding
- ✓ Arithmetic Coding
- ✓ LZW (Lempel–Ziv–Welch) — used in GIF, TIFF, PNG
- ✓ Predictive Coding (DPCM, JPEG-LS, H.264)

GIF: Graphics Interchange Format

JPEG: Joint Photographic Experts Group

DPCM: Differential Pulse Code Modulation

Types/Classification of Image Compression:

B. Lossy Compression

- ✓ Discards less visually important data
- ✓ Human eyes can't notice minor differences
- ✓ Much higher compression ratio

Common Techniques:

- ✓ Transform Coding (DCT in JPEG, DWT, KLT)
- ✓ Vector Quantization
- ✓ Transform + Quantization (DWT — JPEG2000, modern codecs)
- ✓ Fractal Compression
- ✓ Sub-band/Wavelet Coding (JPEG2000, modern video codecs)
- ✓ Hybrid Coding (DCT + Prediction + Entropy) (MPEG, H.264, H.265 (HEVC)).

Predictive Models (Lossless or Lossy)

Applications

Application	Example
Still images	JPEG-LS, PNG filters
Audio compression	DPCM, ADPCM, LPC
Video compression	MPEG, H.264, HEVC (intra/inter prediction)
Remote sensing	Lossless predictive compression for satellites

Predictive Models (Lossless or Lossy)

Example (1D Sequence): Original pixel values,

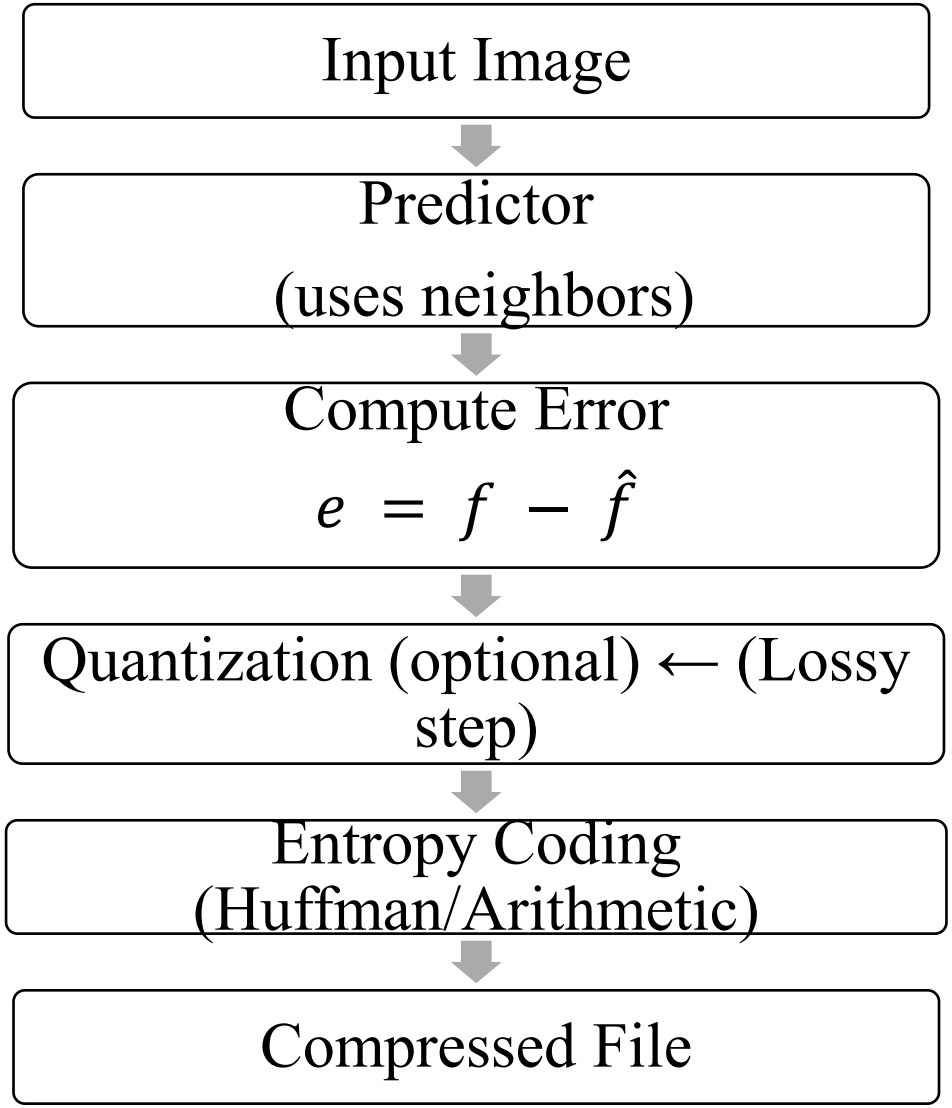
[100, 102, 103, 105, 108]

Predict using previous pixel (A):

Pixel	Predicted	Error (Actual – Predicted)
100	—	—
102	100	+2
103	102	+1
105	103	+2
108	105	+3

Encoded errors: [2, 1, 2, 3] → small values → easy to compress!
If quantized (lossy), these small values can be approximated further.

Predictive Models (Lossless or Lossy)



Predictive Models (Lossless or Lossy)

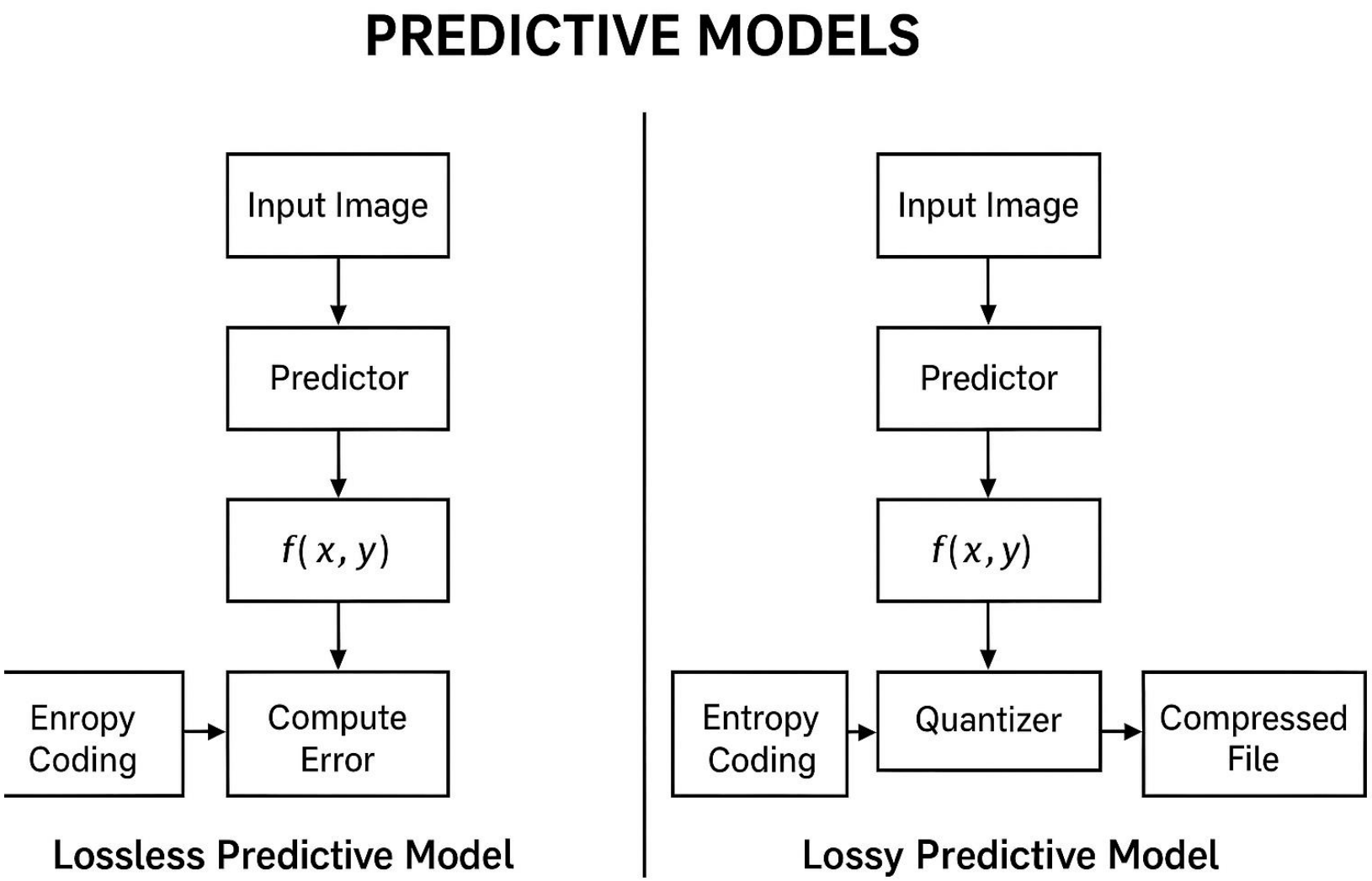
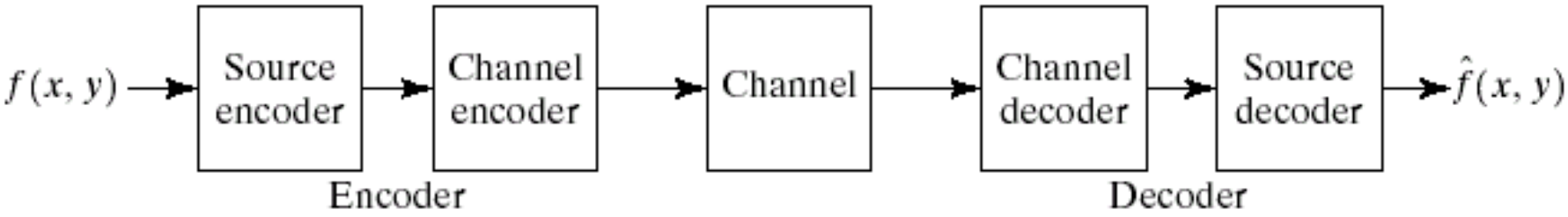
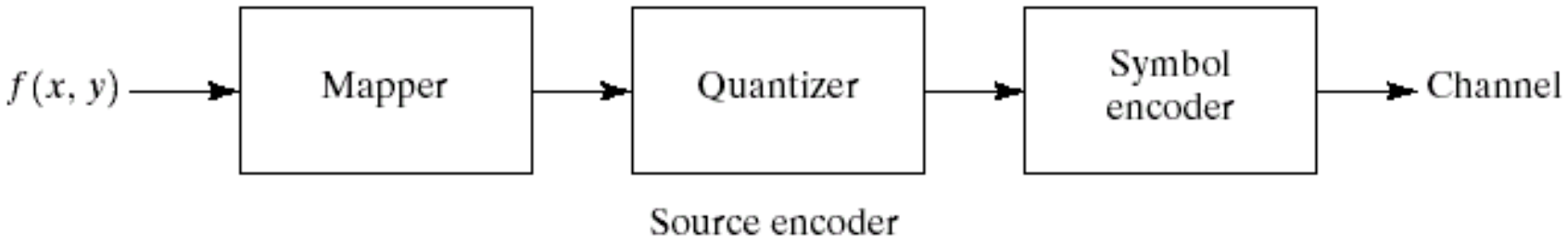


Image Compression Model



Run length JPEG Huffman



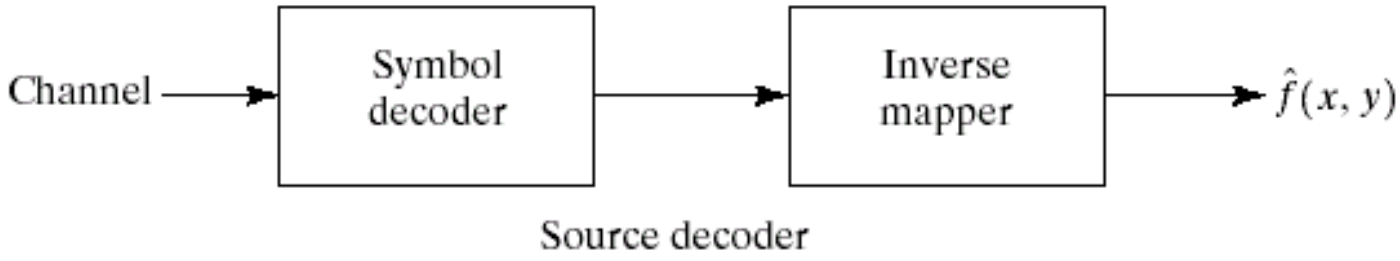
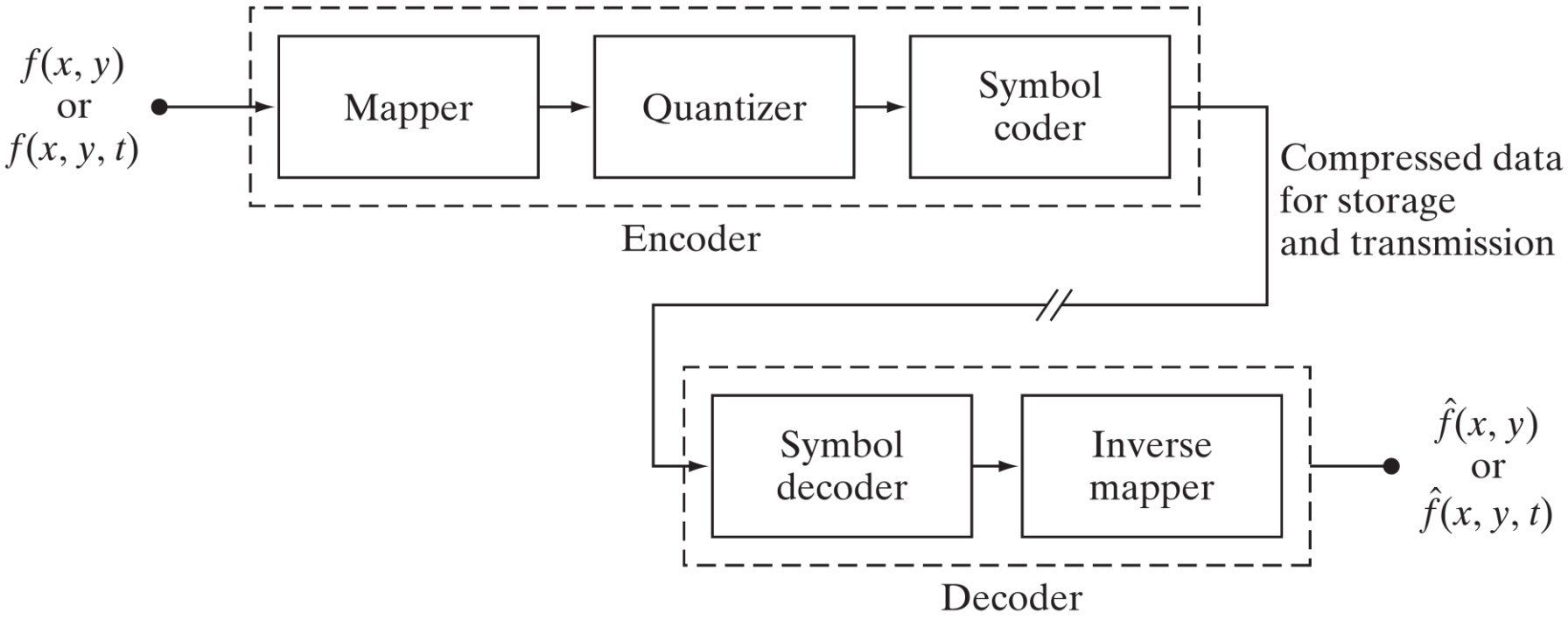


Image Compression Models



Run Length Coding

- Replaces runs of identical pixels with (Run Length value, Pixel count Value).

Example:
AAAAAABBBCC → (A,6)(B,3)(C,2)

Algorithm Steps:

Step 1:
Scan the image **row by row (raster scan order)**.
Step 2:
Count the **number of consecutive identical pixels** (run length).
Step 3:
Store the pair:
(Run Length, Pixel Value)
Step 4:
Repeat until the entire image is encoded.

Run Length Coding

Encoding process:

Pixels	255	255	255	0
Run	3	2	4	2

Encoded output:
(3,255),(2,255),(4,255), (2,0)

Row	1	1	1	0
Run	(3,1)	(4,0)	(2,1)	(2,0)

Encoded output:
(3,1) (4,0) (2,1) (2,0)

Decoding Process

For decoding, simply **expand** each pair (RunLength, Value):
(3,1)→1,1,1 (4,0)→0,0,0,0, etc
Thus, RLE is fully **reversible** (lossless).

Compression Ratio

If the image has long run of identical pixels:

$$CR = \frac{Uncompressed\ Size}{Compressed\ Size}$$

$$\gg 1$$

But if pixel values change frequently (no long runs), RLE can even **increase** file size.

Run Length Coding

Applications

Domain	Example
Fax transmission	Binary text or line drawings
Bitmap (BMP) files	8-bit and 24-bit RLE compression
TIFF, PCX, PDF formats	Optional RLE compression
Medical / Document Scans	Efficient for binary or low-color images

Advantages

- Very simple and fast
- Perfectly lossless
- Effective for images with large uniform regions

Disadvantages

- Inefficient for images with high detail or noise
- Variable-length encoding makes random access harder
- Not suitable for continuous-tone (photographic) images

Run Length Encoding (RLE)

Step 1: Original Image (4×4 pixels)

$$I = \begin{bmatrix} 50 & 50 & 50 & 80 \\ 80 & 80 & 80 & 80 \\ 90 & 90 & 50 & 50 \\ 50 & 50 & 50 & 50 \end{bmatrix}$$

Step 2: Flatten Image by Rows

Write pixels row-wise:

Row 1: 50, 50, 50, 80
Row 2: 80, 80, 80, 80
Row 3: 90, 90, 50, 50
Row 4: 50, 50, 50, 50

Flattened sequence:

50,50,50,80, **80,80,80,80**, 90,90,50,50, **50,50,50,50**

Step 3: Encode with Run-Length Encoding (RLE):

Value	Count
50	3
80	4
90	2
50	6

Run Length Encoding (RLE)

Step 1: Original Image (4×4 pixels)

$$I = \begin{bmatrix} 50 & 50 & 50 & 80 \\ 80 & 80 & 80 & 80 \\ 90 & 90 & 50 & 50 \\ 50 & 50 & 50 & 50 \end{bmatrix}$$

Step 3: Encode with Run-Length Encoding (RLE):

Value	Count
50	3
80	4
90	2
50	6

Step 4: Compressed Representation

Instead of storing all 16 pixels, store:

(50,3), (80,4), (90,2), (50,6)

This is **8 numbers instead of 16**, effectively compressing the image.

Coding Redundancy

$$p_r(r_k) = \frac{n_k}{MN} \qquad k = 0, 1, 2, \ldots, L-1$$

Image size

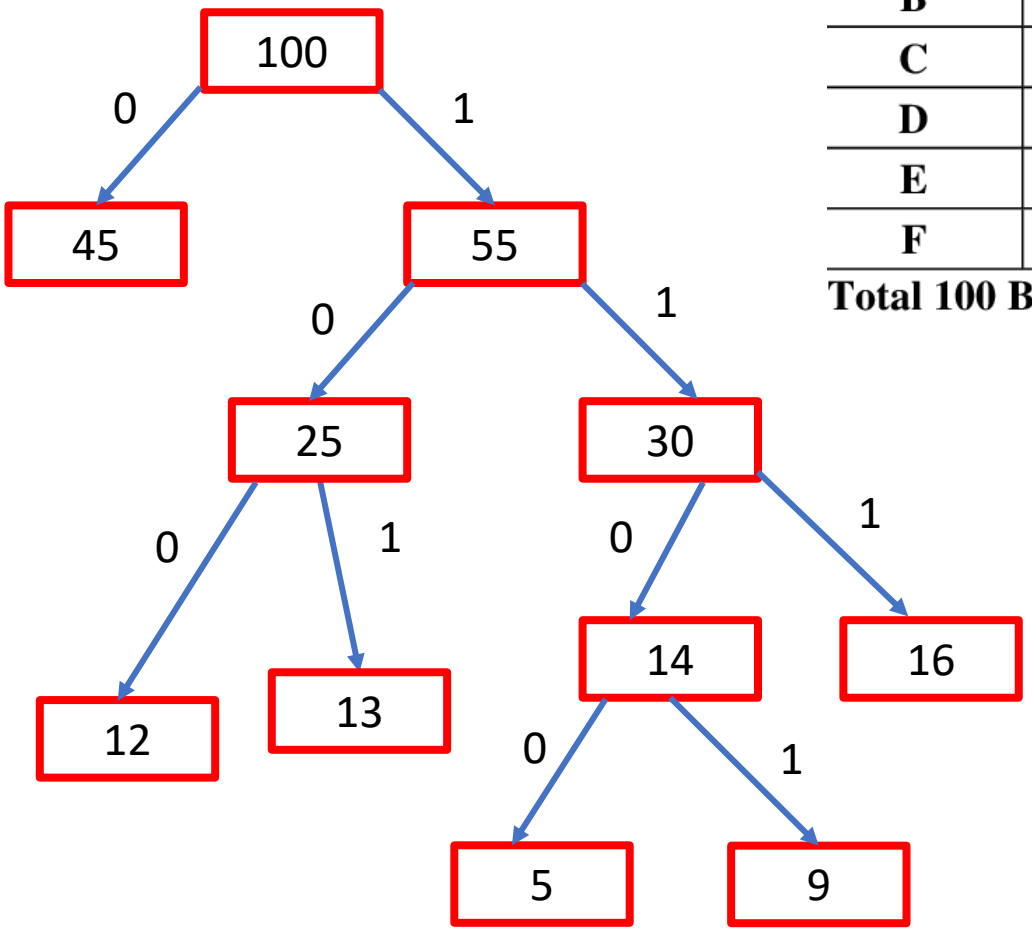
No. of image intensities

Present values

$$L_{\text{avg}} = \sum_{k=0}^{L-1} l(r_k) p_r(r_k)$$

Total number of $M \times N$ *image*: MNL_{avg}

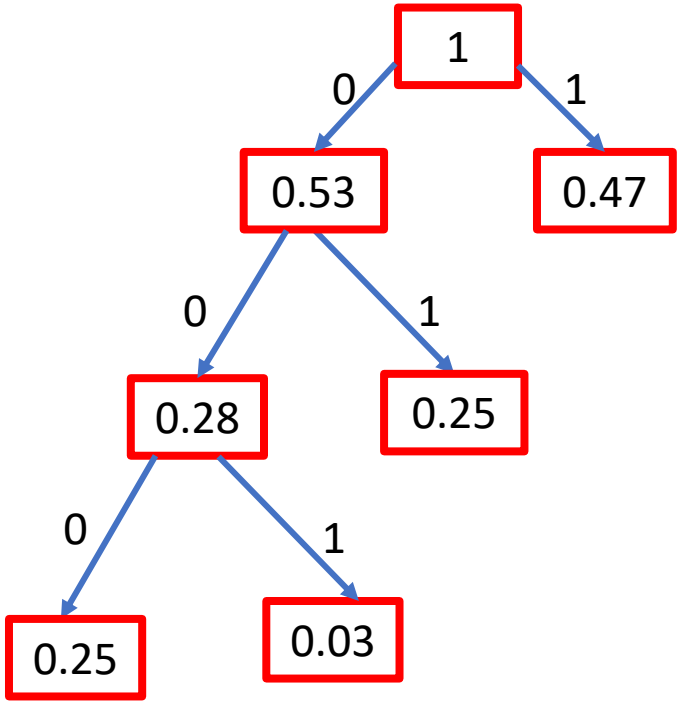
Example of Variable Length Coding



Symbols	Frequency	Variable	
A	45	0	45 × 1 = 45
B	13	101	13 × 3 = 39
C	12	100	12 × 3 = 36
D	16	111	16 × 3 = 48
E	9	1101	09 × 4 = 36
F	5	1100	05 × 4 = 20
Total 100 B (uncompressed)			**224 Bits**

Example of Variable Length Coding

r_k	$p_r(r_k)$	Code 1	$l_1(r_k)$	Code 2	$l_2(r_k)$
$r_{87} = 87$	0.25	01010111	8	01	2
$r_{128} = 128$	0.47	10000000	8	1	1
$r_{186} = 186$	0.25	11000100	8	000	3
$r_{255} = 255$	0.03	11111111	8	001	3
r_k for $k \neq 87, 128, 186, 255$	0	—	8	—	0



Example of Variable Length Coding

r_k	$p_r(r_k)$	Code 1	$l_1(r_k)$	Code 2	$l_2(r_k)$
$r_{87} = 87$	0.25	01010111	8	01	2
$r_{128} = 128$	0.47	10000000	8	1	1
$r_{186} = 186$	0.25	11000100	8	000	3
$r_{255} = 255$	0.03	11111111	8	001	3
r_k for $k \neq 87, 128, 186, 255$	0	—	8	—	0

$$L_{\text{avg}} = \sum_{k=0}^{L-1} l(r_k)p_r(r_k) = 0.25(2) + 0.47(1) + 0.25(3) + 0.03(3) = 1.81 \text{ bits}$$

MNL_{avg}= $256 \times 256 \times 1.81$ or 118,621

Example of Variable Length Coding

r_k	$p_r(r_k)$	Code 1	$l_1(r_k)$	Code 2	$l_2(r_k)$
$r_{87} = 87$	0.25	01010111	8	01	2
$r_{128} = 128$	0.47	10000000	8	1	1
$r_{186} = 186$	0.25	11000100	8	000	3
$r_{255} = 255$	0.03	11111111	8	001	3
r_k for $k \neq 87, 128, 186, 255$	0	—	8	—	0

$$C = \frac{256 \times 256 \times 8}{118{,}621} = \frac{8}{1.81} \approx 4.42 \qquad R = 1 - \frac{1}{4.42} = 0.774$$

Thus, 77.4% of the data in the original 8-bit 2-D intensity array is redundant.

Compression Methods: Huffman Coding

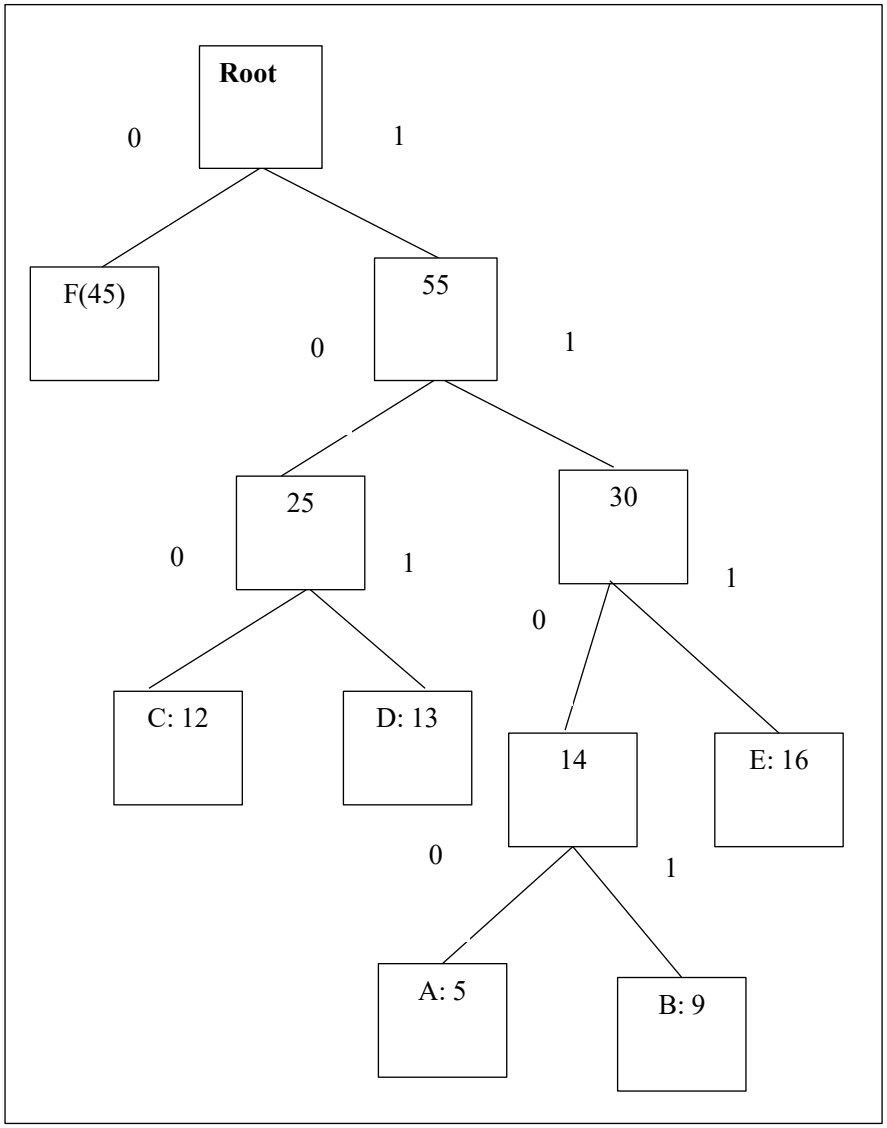
Symbol	A	B	C	D	E	F
Frequency	5	9	12	13	16	45

Step 1: Sort by Frequency

A(5), B(9), C(12), D(13), E(16), F(45)

Step 2: Build Tree

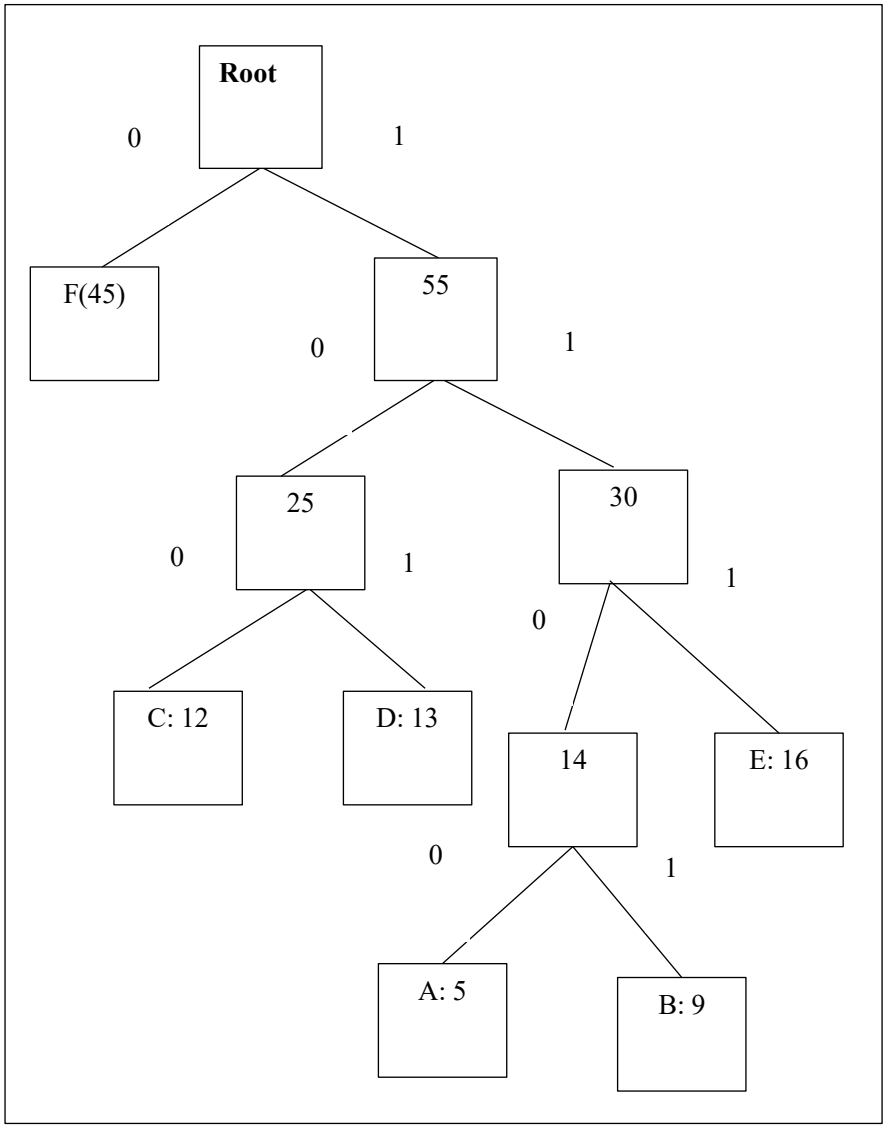
1. Combine A (5) + B (9) → Node1(14)
2. Combine C (12) + D (13) → Node2(25)
3. Combine Node1(14) + E (16) → Node3(30)
4. Combine Node2(25) + Node3(30) → Node4(55)
5. Combine F (45) + Node4(55) → Root (100)



Compression Methods: Huffman Coding

Step 3: Assign Codes

Symbol	Code	Code Length
F	0	1
C	100	3
D	101	3
A	1100	4
B	1101	4
E	111	3



Compression Methods: Huffman Coding

Symbol	Code	Code Length
F	0	1
C	100	3
D	101	3
A	1100	4
B	1101	4
E	111	3

Step 4: Encoded Stream

For the sequence: **"ABACD"**

→ A(1100) B(1101) A(1100) C(100) D(101)

→ Encoded bits: **1100 1101 1100 100 101**→18 bits

Original (ASCII) for **"ABACD"**: 5 × 8 = 40 bits

Compressed: 18 bits

Compression Ratio = 40/18 ≈ 2.22:1

Compression Methods: Huffman Coding

Advantages

- Optimal among all **prefix codes**
- Fully **lossless**
- Simple decoding using binary tree
- Works well with **non-uniform symbol probabilities**

Disadvantages

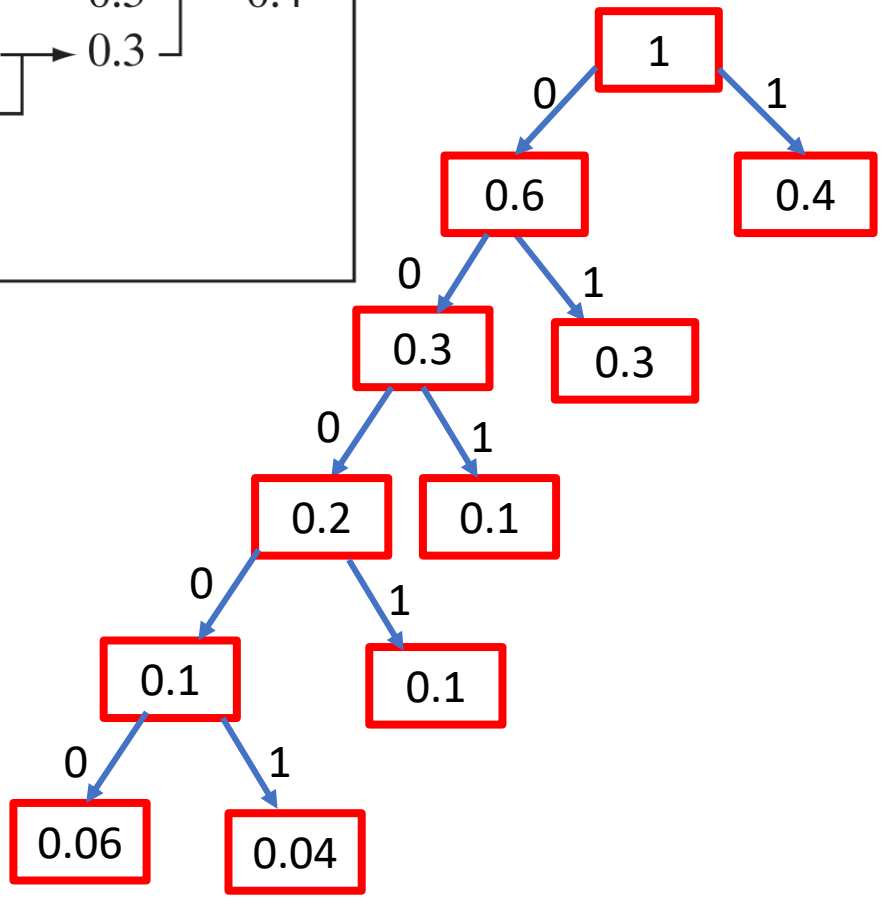
- Needs symbol frequency table → overhead
- Not adaptive to changing data unless recomputed
- Inefficient for uniformly distributed data

Applications

Domain	Example
JPEG	After DCT, coefficients are Huffman encoded
MPEG / MP3	Used for entropy coding
ZIP / GZIP	Part of Deflate algorithm
Fax / Text files	Lossless compression

Compression Methods: Huffman Coding

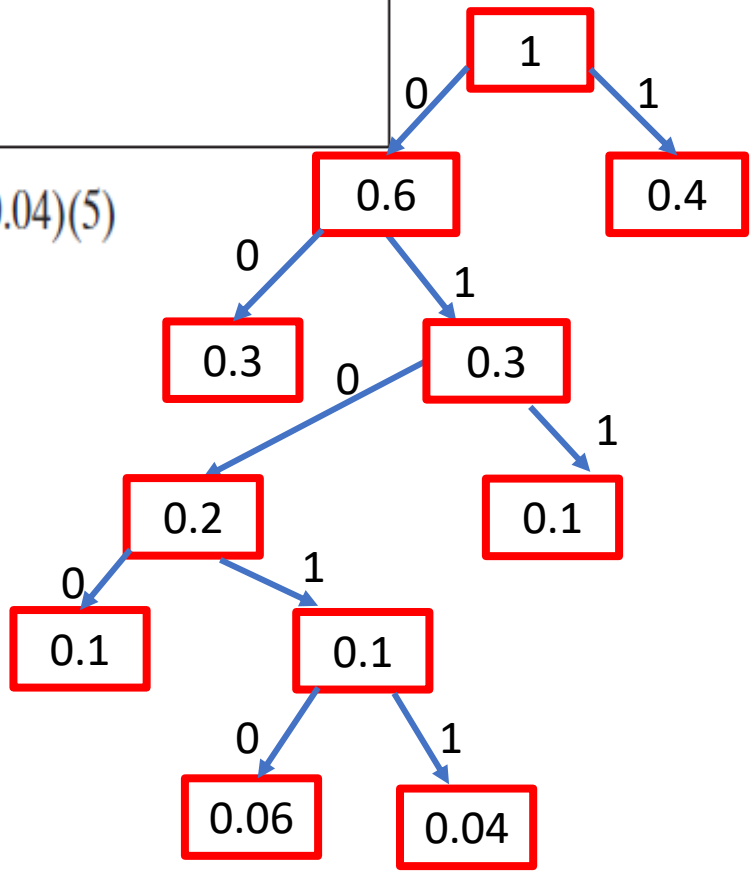
Original source		Source reduction			
Symbol	Probability	1	2	3	4
a_2	0.4	0.4	0.4	0.4	0.6
a_6	0.3	0.3	0.3	0.3	0.4
a_1	0.1	0.1	0.2	0.3	
a_4	0.1	0.1	0.1		
a_3	0.06	0.1			
a_5	0.04				



Compression Methods: Huffman Coding

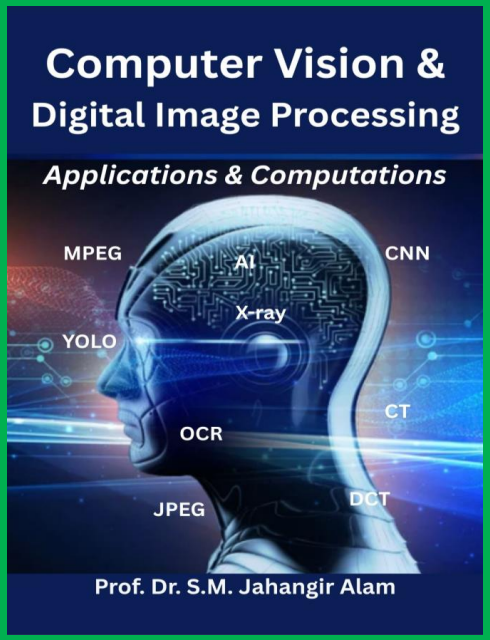
Original source			Source reduction							
Symbol	Probability	Code	1		2		3		4	
a_2	0.4	1	0.4	1	0.4	1	0.4	1	0.6	0
a_6	0.3	00	0.3	00	0.3	00	0.3	00	0.4	1
a_1	0.1	011	0.1	011	0.2	010	0.3	01		
a_4	0.1	0100	0.1	0100	0.1	011				
a_3	0.06	01010	0.1	0101						
a_5	0.04	01011								

$$L_{avg} = (0.4)(1) + (0.3)(2) + (0.1)(3) + (0.1)(4) + (0.06)(5) + (0.04)(5)$$
$$= 2.2 \text{ bits/pixel}$$



Lecture 7.2

Image Compression



Instructor Name:

Prof. Dr. Jahangir Alam

Dept. of CSE, BAUST

Arithmetic Coding

Algorithm Steps

Step	Operation
1. Initialize range	Set Low = 0, High = 1.
2. Divide the interval	Based on symbol probabilities.
3. For each symbol:	Update range: Range = High – Low High = Low + Range × High(symbol) Low = Low + Range × Low(symbol)
4. Output	Any number between final [Low, High) uniquely encodes the message.

Arithmetic Coding

Let's encode **"AB"** with these symbol probabilities:

Step 1: Start

Low = 0, High = 1

Symbol	Probability	Cumulative Range
A	0.6	[0.0, 0.6)
B	0.3	[0.6, 0.9)
C	0.1	[0.9, 1.0)

Step 2: Encode "A"

- Range = 1 – 0 = 1
- A's range = [0.0, 0.6)
- Update:
  - Low = 0 + 1 × 0.0 = 0.0
  - High = 0 + 1 × 0.6 = 0.6

New interval: **[0.0, 0.6)**

Step 3: Encode "B"

- Range = 0.6 – 0.0 = 0.6
- B's range = [0.6, 0.9)
- Update:
  - ✓ Low = 0.0 + 0.6 × 0.6 = 0.36
  - ✓ High = 0.0 + 0.6 × 0.9 = 0.54

Final interval: **[0.36, 0.54)**

Arithmetic Coding

Final Encoded Value

Any value within [0.36, 0.54) represents the message "AB".

For example, choose **0.45**.

5. Decoding Process

Reverse process:

1. Start with encoded number 0.45
2. Find which subrange it belongs to → symbol "A"
3. Update to subrange [0.0, 0.6)
4. Scale 0.45 → new range → falls in "B" subrange

Decode → **"AB"**

Arithmetic Coding

Advantages

- ✓ Achieves near-entropy compression efficiency
- ✓ Works well with fractional probabilities
- ✓ No need for integer-length codes
- ✓ Excellent for adaptive models

Disadvantages

- ✓ Computationally complex (needs floating-point precision)
- ✓ Patent restrictions (historically limited adoption)
- ✓ Requires probability model consistency for encode/decode

Applications

Application	Example
JPEG2000	Image compression
H.264 / HEVC	Video compression
BPG, AV1	Modern codecs
Text compression	Statistical models

Compression Methods: Golomb Coding

Geometrically distributed with *probability mass function* (PMF):

$$P(n) = (1-\rho)\rho^n \qquad 0 < \rho < 1$$

Shortest average code length:

$$m = \left\lceil \frac{\log_2(1+\rho)}{\log_2(1/\rho)} \right\rceil$$

Compression Methods: Golomb Coding

Step 1: Let's take an example with $M = 4 \; and \; N = 6$.

Step 2: Compute $q = \left\lfloor \frac{N}{M} \right\rfloor$. Form the unary code of quotient $\lfloor N/M \rfloor$. The unary code of an integer q is defined as q 1s followed by a 0.

Encode q in unary code → q zeros followed by a 1→ q'.

Step 3: Compute $r = N \mod M$

Step 4: $k = \lceil log_2^M \rceil$, $\therefore c = 2^k - M$

Step 5: Encode r in **truncated binary code** (a slightly shorter form of binary to keep codes compact). Compute truncated remainder r' as

$$r' = \begin{cases} r \; truncated \; to \; k-1 \; bits & 0 \le r < c \\ r + c \; truncated \; to \; k \; bits & otherwise \end{cases}$$

Step 6. Concatenate both parts → final Golomb code. The Golomb code,

G_M (N)=Unary(q') | Binary(r)

Compression Methods: Golomb Coding

Example3 (M = 2):

Let, N=9, M=2, $q = \lfloor 9/2 \rfloor = 4, r = 9 \; mod \; 2 = 1$

Unary for $q = 4$: $00001 \rightarrow 11110$

$M = 2 \rightarrow$ needs truncated binary code:

- $k = \lceil log_2^M \rceil = \lceil log_2^2 = 1 \rceil$
- First $c = 2^k - M = 2^1 - 2 = 0$ remainders $use \; k = 0$ bits
- The rest use $r < c \; thus \; r \; truncated \; to \; k - 1 = 0$ bits

Since, $r = 1 \geq 1$, Binary of $r = 1$ for $k = 1$ bits: 1

Golomb Code, G_2 (9): 11110 |1 $\rightarrow$ 111101

Compression Methods: Golomb Coding

$G_1\ (0)$:$\lfloor 0/1 \rfloor = \lfloor 0 \rfloor = 0$

$G_1\ (1)$:$\lfloor 1/1 \rfloor = \lfloor 1 \rfloor = 1$

$G_1\ (2)$:$\lfloor 2/1 \rfloor = \lfloor 2 \rfloor = 2$

$G_1\ (3)$:$\lfloor 3/1 \rfloor = \lfloor 3 \rfloor = 3$

$G_1\ (4)$:$\lfloor 4/1 \rfloor = \lfloor 4 \rfloor = 4$

$G_1\ (5)$:$\lfloor 5/1 \rfloor = \lfloor 5 \rfloor = 5$

$G_1\ (6)$:$\lfloor 6/1 \rfloor = \lfloor 6 \rfloor = 6$

$G_1\ (7)$:$\lfloor 7/1 \rfloor = \lfloor 7 \rfloor = 7$

$G_1\ (8)$:$\lfloor 8/1 \rfloor = \lfloor 8 \rfloor = 8$

$G_1\ (9)$:$\lfloor 9/1 \rfloor = \lfloor 9 \rfloor = 9$

$G_m\ (n)$

n	$G_1(n)$	$G_2(n)$	$G_4(n)$	$G^0_{\exp}(n)$
0	0	00	000	0
1	10	01	001	100
2	110	100	010	101
3	1110	101	011	11000
4	11110	1100	1000	11001
5	111110	1101	1001	11010
6	1111110	11100	1010	11011
7	11111110	11101	1011	1110000
8	111111110	111100	11000	1110001
9	1111111110	111101	11001	1110010

Compression Methods: Golomb Coding

$G_2\,(0)\text{:}\lfloor 0/2 \rfloor = \lfloor 0 \rfloor = 0$

$G_2\,(1)\text{:}\lfloor 1/2 \rfloor = \lfloor 0.5 \rfloor = 0$

$G_2\,(2)\text{:}\lfloor 2/2 \rfloor = \lfloor 1 \rfloor = 1$

$G_2\,(3)\text{:}\lfloor 3/2 \rfloor = \lfloor 1.5 \rfloor = 1$

$G_2\,(4)\text{:}\lfloor 4/2 \rfloor = \lfloor 2 \rfloor = 2$

$G_2\,(5)\text{:}\lfloor 5/2 \rfloor = \lfloor 2.5 \rfloor = 2$

$G_2\,(6)\text{:}\lfloor 6/2 \rfloor = \lfloor 3 \rfloor = 3$

$G_2\,(7)\text{:}\lfloor 7/2 \rfloor = \lfloor 3.5 \rfloor = 3$

$G_2\,(8)\text{:}\lfloor 8/2 \rfloor = \lfloor 4 \rfloor = 4$

$G_2\,(9)\text{:}\lfloor 9/2 \rfloor = \lfloor 4.5 \rfloor = 4$

n	$G_1(n)$	$G_2(n)$	$G_4(n)$	$G^0_{\exp}(n)$
0	0	00	000	0
1	10	01	001	100
2	110	100	010	101
3	1110	101	011	11000
4	11110	1100	1000	11001
5	111110	1101	1001	11010
6	1111110	11100	1010	11011
7	11111110	11101	1011	1110000
8	111111110	111100	11000	1110001
9	1111111110	111101	11001	1110010

Compression Methods: Golomb Coding

n	$G_1(n)$	$G_2(n)$	$G_4(n)$	$G^0_{\exp}(n)$
0	0	00	000	0
1	10	01	001	100
2	110	100	010	101
3	1110	101	011	11000
4	11110	1100	1000	11001
5	111110	1101	1001	11010
6	1111110	11100	1010	11011
7	11111110	11101	1011	1110000
8	111111110	111100	11000	1110001
9	1111111110	111101	11001	1110010

$k = \lceil log_2^4 \rceil = 2$

$c = 2^2 - 4 = 0$

r= $9 mod 4$
$= 1001 mod 0100$
=0001

r' = 01

r truncated to 2 bits of *r*,

G_4 (9):$\lfloor 9/4 \rfloor = \lfloor 2.25 \rfloor = 2$

G_4 (9): 110 01

Compression Methods: Golomb Coding

n	$G_1(n)$	$G_2(n)$	$G_4(n)$	$G^0_{\exp}(n)$
0	0	00	000	0
1	10	01	001	100
2	110	100	010	101
3	1110	101	011	11000
4	11110	1100	1000	11001
5	111110	1101	1001	11010
6	1111110	11100	1010	11011
7	11111110	11101	1011	1110000
8	111111110	111100	11000	1110001
9	1111111110	111101	11001	1110010

$k = \lceil log_2^4 \rceil = 2$

$c = 2^2 - 4 = 0$

r= $7 mod 4$
$= 0111 mod 0100$
=0011

r truncated to 2 bits of r,
$r' = 11$

G_4 (7): 10 11

G_4 (7):$\lfloor 7/4 \rfloor = \lfloor 1.75 \rfloor = 1$

Compression Methods: Golomb Coding

$G_4\ (0):\lfloor 0/4 \rfloor = \lfloor 0 \rfloor = 0$

$G_4\ (1):\lfloor 1/4 \rfloor = \lfloor 0.25 \rfloor = 0$

$G_4\ (2):\lfloor 2/4 \rfloor = \lfloor 0.5 \rfloor = 0$

$G_4\ (3):\lfloor 3/4 \rfloor = \lfloor 0.75 \rfloor = 0$

$G_4\ (4):\lfloor 4/4 \rfloor = \lfloor 1 \rfloor = 1$

$G_4\ (5):\lfloor 5/4 \rfloor = \lfloor 1.25 \rfloor = 1$

$G_4\ (6):\lfloor 6/4 \rfloor = \lfloor 1.5 \rfloor = 1$

$G_4\ (7):\lfloor 7/4 \rfloor = \lfloor 1.75 \rfloor = 1$

$G_4\ (8):\lfloor 8/4 \rfloor = \lfloor 2 \rfloor = 2$

$G_4\ (9):\lfloor 9/4 \rfloor = \lfloor 2.25 \rfloor = 2$

n	$G_1(n)$	$G_2(n)$	$G_4(n)$	$G^0_{\exp}(n)$
0	0	00	000	0
1	10	01	001	100
2	110	100	010	101
3	1110	101	011	11000
4	11110	1100	1000	11001
5	111110	1101	1001	11010
6	1111110	11100	1010	11011
7	11111110	11101	1011	1110000
8	111111110	111100	11000	1110001
9	1111111110	111101	11001	1110010

Exponential Golomb Coding

Order-k exponential-Golomb code, $G_{exp}^{k}(n)$:

Step 1: Find an integer $i \geq 0$ such that

$$\sum_{j=0}^{i-1} 2^{j+k} \leq n < \sum_{j=0}^{i} 2^{j+k}$$

and form the unary code of i.

If $k = 0, i = \lfloor log_2(n+1) \rfloor$ and the code is also known as the ***Elias gamma code***.

Step 2: Truncate the binary representation of

$$n - \sum_{j=0}^{i-1} 2^{j+k}$$

to $k + i$ least significant bits.

Step 3: Concatenate the results of steps 1 and 2.

Exponential Golomb Coding

1. $If\ k = 0, n = 8\ then\ i = \lfloor log_2(n+1) \rfloor = \lfloor log_2(8+1) \rfloor = 3$

The unary code of i is 1110

2.

$$\sum_{j=0}^{3-1} 2^{j+0} \leq 8 < \sum_{j=0}^{3} 2^{j+0} \Rightarrow \sum_{j=0}^{2} 2^{j} \leq 8 < \sum_{j=0}^{3} 2^{j}$$

$$\Rightarrow 2^0 + 2^1 + 2^2 \leq 8 < 2^0 + 2^1 + 2^2 + 2^3 \Rightarrow 7 \leq 8 < 15$$

3. The binary of

$$8 - \sum_{j=0}^{3-1} 2^{j+0} = 8 - \sum_{j=0}^{2} 2^{j} = 8 - 7 = 1 = 0001$$

4. $k + i = 3 + 0$ least significant bits become 001

5. Concatenate the results of steps 1 and 4 gives: 1110 001

Compression Methods: Golomb Coding

Contains the first 10 codes of the zeroth order *exponential-Golomb code*

$$G^0_{\exp}(n)$$

G^0_{exp} (8)

n	$G_1(n)$	$G_2(n)$	$G_4(n)$	$G^0_{\exp}(n)$
0	0	00	000	0
1	10	01	001	100
2	110	100	010	101
3	1110	101	011	11000
4	11110	1100	1000	11001
5	111110	1101	1001	11010
6	1111110	11100	1010	11011
7	11111110	11101	1011	1110000
8	111111110	111100	11000	1110001
9	1111111110	111101	11001	1110010

Dictionary-based (LZW)

Dictionary-Based Compression, focusing on the most famous algorithm: LZW (Lempel–Ziv–Welch) — a lossless compression technique used in GIF, TIFF, and PDF formats.

Applications

Format	Where used
GIF	Image compression
TIFF	Tagged Image File Format
PDF	Stream compression
Unix “compress” utility	File compression

Algorithm Steps (Encoding): LZW

Step 1:

Initialize the dictionary with **all possible symbols** of the source alphabet. (e.g., for 8-bit images: entries 0–255).

Step 2:

Let W = first input symbol.

Step 3:

While there are symbols left:

1. Read next symbol **K**.
2. If **W+K** exists in dictionary →**W = W+K**.
3. Else →
   - ✓ Output the **code for W**.
   - ✓ Add **W+K** to the dictionary with a new code.
   - ✓ Set **W = K**.

Step 4:

Output code for **W**.

Dictionary-based (LZW)

Example: Message,
A B B A A B AB A A B AB

Step 1: Initialize Dictionary
Next available code starts at **256**.

Code	Entry
65	A
66	B

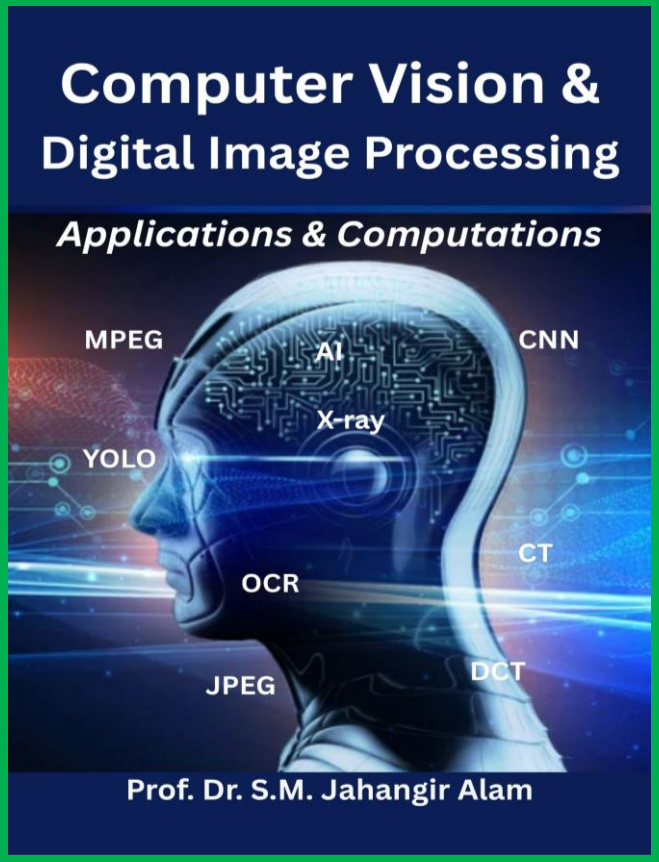
Step 2: Encoding Steps

Step	W	K	W+K	Output	New Entry	New Code
1	A	B	AB	—	AB	256
2	B	A	BA	65	BA	257
3	A	B	AB (exists)	—	—	—
4	AB	A	ABA	256	ABA	258
5	A	B	AB (exists)	—	—	—
6	AB	—	—	256	—	—

Encoded Output: 65, 257, 256, 256

Lecture 7.3

Image Compression



Instructor Name:

Prof. Dr. Jahangir Alam

Dept. of CSE, BAUST

Discrete Cosine Transform Algorithm

- ✓**DCT** is used in lossy image compression because it has very strong energy compaction, i.e., its large amount of information is stored in very low frequency component of a signal and rest other frequency having very small data which can be stored by using very less number of bits (usually, at most 2 or 3 bit).
- ✓For DCT Transformation on an image, first, have to fetch image file information (pixel value in term of integer having range 0-255) which divides in block of 8X8 matrix & then apply DCT on that block of data.

Discrete Cosine Transform Algorithm

The DCT-II formula,

$$1D:\ F(u) = \alpha(u) \sum_{n=0}^{N-1} f(x)\, cos\left[\frac{\pi(2x+)u}{2N}\right], \qquad u = 0,1,2,3$$

$$2D:\ F(u,v)$$

$$= \frac{1}{2}\alpha(u)\alpha(v) \sum_{x=0}^{N-1}\sum_{y=0}^{N-1} f(x,y)\, cos\left[\frac{\pi(2x+1)u}{2N}\right] cos\left[\frac{\pi(2y+1)v}{2N}\right]$$

Image Transform Coding (JPEG Compression)

52	55
53	54

Step 1: Apply 2×2 Discrete Cosine Transform (DCT)

$$C = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

DCT formula for 2×2 block *X*: $Y = C \cdot X \cdot C^T$

$$C \cdot X = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}\begin{bmatrix} 52 & 55 \\ 53 & 54 \end{bmatrix} = \begin{bmatrix} 52.5 & 54.5 \\ -0.5 & 0.5 \end{bmatrix}$$

$$Y = C \cdot X \cdot C^T = \begin{bmatrix} 52.5 & 54.5 \\ -0.5 & 0.5 \end{bmatrix}\frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}^T$$

$$= \frac{1}{2}\begin{bmatrix} 52.5 & 54.5 \\ -0.5 & 0.5 \end{bmatrix}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} 53.5 & -1 \\ 0 & -0.5 \end{bmatrix}$$

Image Transform Coding (JPEG Compression)

52	55
53	54

$$Y = C \cdot X \cdot C^T = \begin{bmatrix} 53.5 & -1 \\ 0 & -0.5 \end{bmatrix}$$

Step 3: Interpretation of coefficients Y

Most energy is in the DC coefficient, higher-frequency terms are small.

Coefficient	Value	Meaning
(1,1)DC	53.5	DC coefficient (average)
(1,2)	-1	Horizontal frequency
(2,1)DC	0	Vertical frequency
(2,2)DC	-0.5	Diagonal frequency

Image Transform Coding (JPEG Compression)

$$Y = C \cdot X \cdot C^T = \begin{bmatrix} 53.5 & -1 \\ 0 & -0.5 \end{bmatrix}$$

Step 4: Quantization (simplified)

Assume quantization step size = 2:

$$Q_{ij} = round\left(\frac{(Y_{ij})}{2}\right)$$

$$Quantized\ coefficients:$$

$$Q = \begin{bmatrix} round(53.5/2) & round(-1/2) \\ round(0/2) & round(-0.5/2) \end{bmatrix} = \begin{bmatrix} 27 & 0 \\ 0 & 0 \end{bmatrix}$$

Step 5: Compression

- ✓ Store quantized coefficients instead of raw pixel values.
- ✓ DC coefficient stores most image info.
- ✓ High-frequency coefficients often become zero, allowing efficient compression.

Image Transform Coding (JPEG Compression)

Step 6: Reconstruction (Inverse DCT)

Using quantized coefficients Q, multiply back by inverse transform to get approximate image block.

Summary:

Step	Description
Input Image	2×2 pixel block
Transform	Apply DCT to convert spatial → frequency domain
Quantization	Reduce precision of coefficients
Compression	Store quantized coefficients efficiently
Reconstruction	Approximate image via inverse transform

Transform-based Encoding using 1-D DCT

A simple example of Transform-Based Encoding for image compression, using the Discrete Cosine Transform (DCT) — a classic method used in JPEG.

1-D $x = [52, 55, 61, 66]$

Step 1: Compute the 1-D DCT of the signal

The formula for DCT type-II for *N*=4 samples:

$$X_k = \alpha(k) \sum_{n=0}^{N-1} x_n \cos\left[\frac{\pi}{N}\left(n + \frac{1}{2}\right)k\right], \qquad k = 0,1,2,3$$

$$\alpha(k) = \begin{cases} \sqrt{\frac{1}{N}} & if\ k = 0 \\ \sqrt{\frac{2}{N}} & k \geq 1 \end{cases}$$

N=4, For *k*=0:

$$\alpha(0) = \sqrt{1/4} = \frac{1}{2}$$

$$X_0 = \frac{1}{2}\sum_{n=0}^{3} x_n \cos\left[\frac{\pi}{4}(n + 0.5) \times 0\right] = \frac{1}{2}(52 + 55 + 61 + 66) = 117$$

Transform-based Encoding using 1-D DCT

$$\alpha(k) = \begin{cases} \sqrt{\frac{1}{N}} & if\ k = 0 \\ \sqrt{\frac{2}{N}} & k \geq 1 \end{cases}$$

N=4, For k=1:

$$\alpha(0) = \sqrt{2/4} = \frac{1}{\sqrt{2}} = 0.707$$

$$X_1 = \frac{1}{\sqrt{2}} \sum_{n=0}^{3} x_n \cos\left[\frac{\pi}{4}(n + 0.5) \times 1\right]$$

n	$(n + 0.5)\pi/4$ (radians)	cos(…)
0	$(0 + 0.5) \times \pi/4 = \pi/8 \approx 0.393$	$cos(0.393) = 0.924$
1	$1.5 \times \pi/4 = 3\pi/8 \approx 1.178$	$cos(1.178) = 0.382$
2	$2.5 \times \pi/4 = 5\pi/8 \approx 1.963$	$cos(1.963) = -0.382$
3	$3.5 \times \pi/4 = 7\pi/8 \approx 2.748$	$cos(2.748) = -0.924$

$$X_1 = \frac{1}{\sqrt{2}}[\mathbf{52 \times 0.924 + 55 \times 0.382 + 61 \times (-0.382) + 66 \times (-0.924)}]$$
$$= \mathbf{0.707 \times -15.22 = -10.76}$$

Transform-based Encoding using 1-D DCT

k=2:

$$X_2 = \frac{1}{\sqrt{2}} \sum_{n=0}^{3} x_n \cos\left[\frac{\pi}{4}(n+0.5) \times 2\right]$$

n	$(n+0.5)\pi/4$ (radians)	cos(…)
0	$2(0+0.5) \times \pi/4 = \pi/4 \approx 0.785$	$\cos(0.785) = 0.707$
1	$2(1.5 \times \pi/4) = 3\pi/4 \approx 2.356$	$\cos(2.356) = 0.707$
2	$2(2.5 \times \frac{\pi}{4}) = 5\pi/4 \approx 3.927$	$\cos(3.927) = -0.707$
3	$2(3.5 \times \frac{\pi}{4}) = 7\pi/4 \approx 5.498$	$\cos(2.748) = -0.707$

$$X_2 = \frac{1}{\sqrt{2}}\left[52 \times 0.707 + 55 \times (-0.707) + 61 \times (-0.707) + 66 \times 0.707\right] = 1.40 = .707 \times 1.40 = 0.99$$

Transform-based Encoding using 1-D DCT

k=3:

$$X_3 = \frac{1}{\sqrt{2}} \sum_{n=0}^{3} x_n \cos\left[\frac{\pi}{4}(n + 0.5) \times 3\right]$$

X_3
$= 0.707$
$\times [52 \times 0.382 + 55 \times (-0.924) + 61 \times 0.924 + 66 \times (-0.382)]$
$= 0.707 \times 0.19 = 0.13$

Step 2: Resulting DCT Coefficients

$$X = [117, -10.76, 0.99, 0.13]$$

Step 3: Quantization (example)

Assume we quantize coefficients by dividing by a quantization factor *Q*=10 and rounding:

$$QX = round\left(\frac{X}{10}\right) = [12, -1, 0, 0]$$

Transform-based Encoding using 1-D DCT

Step 4: Encoding

Only keep significant coefficients:

- 12 (DC component)
- -1 (1st AC component)

The others are zero and can be encoded efficiently.

Step 5: Reconstruction (optional)

Multiply quantized coefficients back:

$$\hat{X} = QX \times Q = [120, -10, 0, 0]$$

Perform inverse DCT (similar formula) to get approximate reconstructed signal.

JPEG-style compression (DCT and quantization)

Step 0. Example Input 2×2 Block

$$f(x,y) = \begin{bmatrix} 100 & 120 \\ 80 & 90 \end{bmatrix}$$

Step 1. Level Shift (subtract 128): JPEG always centers pixel values around 0 before applying DCT:

$$f'(x,y) = \begin{bmatrix} 100 & 120 \\ 80 & 90 \end{bmatrix} - 128 = \begin{bmatrix} -28 & -8 \\ -48 & -38 \end{bmatrix}$$

Step 2. DCT-II Formula (for N = 2)

$2D\colon\ F(u,v)$

$$= \frac{1}{2}\alpha(u)\alpha(v)\sum_{x=0}^{1}\sum_{y=0}^{1} f'(x,y)\ cos\left[\frac{\pi(2x+1)u}{4}\right] cos\left[\frac{\pi(2y+1)v}{4}\right]$$

$$\alpha(u) = \begin{cases} \frac{1}{\sqrt{2}} & if\, u = 0 \\ 1 & if\ u > 0 \end{cases}$$

JPEG-style compression (DCT and quantization)

Step 3. Compute All Four Coefficients

(a) DC term F(0,0)

$$F(0,0) = \frac{1}{2}\cdot\frac{1}{\sqrt{2}}\cdot\frac{1}{\sqrt{2}}\sum_{x=0}^{1}\sum_{y=0}^{1} f'(x,y) = \frac{1}{4}[(-28) + (-8) + (-48) + (-38)] = -30.5$$

(b) DC term F(0,1): $\alpha(0) = \frac{1}{\sqrt{2}}, \alpha(1) = 1$

v	y	$cos\left[\frac{\pi(2y+1)}{4}\right]$
1	0	$\frac{\sqrt{2}}{2} = 0.7071$
1	1	$-\frac{\sqrt{2}}{2} = -0.7071$

JPEG-style compression (DCT and quantization)

Thus, $cos\left[\frac{\pi(2y+1)}{4}\right] = [0.7071, -0.7071]$

$$\therefore F(0,1) = \frac{1}{2}.\frac{1}{\sqrt{2}}\sum_{x=0}^{1}\sum_{y=0}^{1} f'(x,y)cos\left[\frac{\pi(2y+1)}{4}\right]$$

$$\Rightarrow F(0,1)$$
$$= \frac{1}{2\sqrt{2}}\left[f'(0,0).\frac{\sqrt{2}}{2} + f'(0,1).\left(-\frac{\sqrt{2}}{2}\right)\right] + \left[f'(1,0).\frac{\sqrt{2}}{2} + f'(1,1).\left(-\frac{\sqrt{2}}{2}\right)\right]$$

$$\Rightarrow F(0,1)$$
$$= \frac{1}{2\sqrt{2}}[(-28)(0.7071) + (-8)(-0.7071) + (-48)(0.7071)$$
$$+ (-38)(-0.7071)] = -7.5$$

JPEG-style compression (DCT and quantization)

(c) AC term F(1,0): By symmetry (vertical variation instead of horizontal). $\alpha(1) = 1, \alpha(0) = \frac{1}{\sqrt{2}}$

u	x	$cos\left[\frac{\pi(2x+1)}{4}\right]$
1	0	$\frac{\sqrt{2}}{2} = 0.7071$
1	1	$-\frac{\sqrt{2}}{2} = -0.7071$

$$\therefore F(1,0)$$

$$= \frac{1}{2}.\frac{1}{\sqrt{2}}\sum_{x=0}^{1}\sum_{y=0}^{1} f'(x,y)cos\left[\frac{\pi(2x+1)}{4}\right]$$

$$\Rightarrow F(0,1)$$

$$= \frac{1}{2\sqrt{2}}\left[f'(0,0).\frac{\sqrt{2}}{2} + f'(0,1).\left(\frac{\sqrt{2}}{2}\right)\right] + \left[f'(1,0).\left(-\frac{\sqrt{2}}{2}\right) + f'(1,1).\left(-\frac{\sqrt{2}}{2}\right)\right]$$

$$F(1,0) = \frac{1}{2\sqrt{2}}[(-28)(0.7071) + (-8)(0.7071) + (-48)(-0.7071) + (-38)(-0.7071)] = 12.5$$

JPEG-style compression (DCT and quantization)

(d) AC term F(1,1): Both cosine factors = [0.7071, −0.7071], thus

$$\alpha(1) = 1, \alpha(1) = 1$$

$$cos\left[\frac{\pi(2t+1)}{4}\right] = \begin{cases} cos\left[\frac{\pi}{4}\right] = \frac{\sqrt{2}}{2} & for\ t = 0 \\ cos\left[\frac{3\pi}{4}\right] = -\frac{\sqrt{2}}{2} & for\ t = 1 \end{cases}$$

The product of the two cosines for each (x, y) is therefore

$$\left(\pm\frac{\sqrt{2}}{2}\right)\left(\pm\frac{\sqrt{2}}{2}\right) = \pm\frac{1}{2}$$

Thus, $(x, y) = \left\{(0,0) = +\frac{1}{2}, (0,1) = -\frac{1}{2}, (1,0) = -\frac{1}{2}, (1,1) = +\frac{1}{2}\right\}$

JPEG-style compression (DCT and quantization)

$$F(1,1) = \frac{1}{2}\sum_{x=0}^{1}\sum_{y=0}^{1} f'(x,y)cos\left[\frac{\pi(2x+1)}{4}\right]cos\left[\frac{\pi(2y+1)}{4}\right]$$

$$\frac{1}{2}\left[(-28)\left(+\frac{1}{2}\right)+(-8)\left(-\frac{1}{2}\right)+(-48)\left(-\frac{1}{2}\right)+(-38)\left(+\frac{1}{2}\right)\right] = -2.5$$

Step 4. Final 2×2 DCT Matrix

$$F(u,v) = \begin{bmatrix} -30.5 & -7.5 \\ 12.5 & -2.5 \end{bmatrix}$$

JPEG-style compression (DCT and quantization)

3) Quantization (very simplified 2×2 quant table)

JPEG uses a quantization matrix. For this toy 2×2 example pick a simple uniform quant table

$$Q = \begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}$$

(Just to illustrate — real JPEG uses 8×8 tables.)

Divide elementwise and round to nearest integer (round halves up):

$$F_{quant} = round\left(\frac{F}{Q}\right) = round\left(\begin{bmatrix} -30.5/10 & -7.5/10 \\ 12.5/10 & -2.5/10 \end{bmatrix}\right)$$

$$= \begin{bmatrix} -3 & 0 \\ 1 & 0 \end{bmatrix}$$

Thus, the smallest AC survive as small integers; DC is compressed from $-30.5 \rightarrow -3$ (coarser).

JPEG-style compression (DCT and quantization)

4) Zig-zag ordering (for 2×2)

A typical zig-zag order for 2×2 is:

$$(0,0), (0,1), (1,0), (1,1)$$

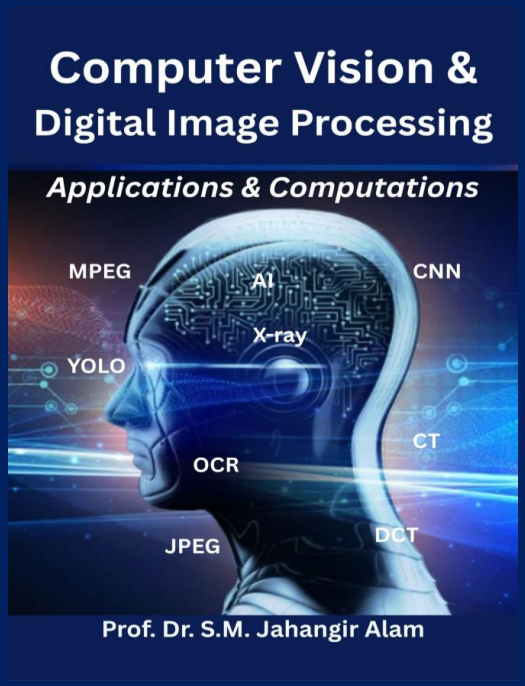
Applying that to F_{quant} gives the sequence:

$$[-6, -2, 3, -1]$$

That sequence would then be run-length encoded and entropy-coded in **JPEG**.

Lecture 7.4

Image Compression



Instructor Name:

Prof. Dr. Jahangir Alam

Dept. of CSE, BAUST

MPEG

##A simplified MPEG video compression illustrates the core ideas: I-frames, P-frames, motion estimation, and residual encoding.

MPEG Basics Recap

- **I-frame (Intra-coded):** Encoded independently, like a JPEG image.
- **P-frame (Predictive-coded):** Encodes differences (motion + residuals) from a previous frame.
- **Motion Estimation:** Finding how blocks move from one frame to the next.
- **Residual:** The difference after compensating for motion.

MPEG

Example Setup

Suppose we have two consecutive 4×4 grayscale frames of a video.

- **Frame 1 (I-frame)**

$$F1 = \begin{bmatrix} 10 & 10 & 20 & 20 \\ 10 & 10 & 20 & 20 \\ 30 & 30 & 40 & 40 \\ 30 & 30 & 40 & 40 \end{bmatrix}$$

Frame 2 (P-frame)

The objects shifted right by 1 pixel:

$$F2 = \begin{bmatrix} 0 & 10 & 10 & 20 \\ 0 & 10 & 10 & 20 \\ 0 & 30 & 30 & 40 \\ 0 & 30 & 30 & 40 \end{bmatrix}$$

Step 1: Encode Frame 1 as I-frame

- Compress F1 as a still image using a method like JPEG (DCT + quantization).
- Store it fully in the bitstream.

MPEG

- **Frame 1 (I-frame)**

$$F1 = \begin{bmatrix} 10 & 10 & 20 & 20 \\ 10 & 10 & 20 & 20 \\ 30 & 30 & 40 & 40 \\ 30 & 30 & 40 & 40 \end{bmatrix}$$

Frame 2 (P-frame)

The objects shifted right by 1 pixel:

$$F2 = \begin{bmatrix} 0 & 10 & 10 & 20 \\ 0 & 10 & 10 & 20 \\ 0 & 30 & 30 & 40 \\ 0 & 30 & 30 & 40 \end{bmatrix}$$

Step 2: Motion Estimation for Frame 2

- ✓ Divide F2 into 2×2 blocks.
- ✓ For each block in $F2$, find the best matching block in $F1$ within a search window.

Blocks in Frame 2

Block	Pixels	Location (row,col)
1	$\begin{bmatrix} 0 & 10 \\ 0 & 10 \end{bmatrix}$	(0,0)
2	$\begin{bmatrix} 10 & 20 \\ 10 & 20 \end{bmatrix}$	(0,2)
3	$\begin{bmatrix} 0 & 30 \\ 0 & 30 \end{bmatrix}$	(2,0)
4	$\begin{bmatrix} 30 & 40 \\ 30 & 40 \end{bmatrix}$	(2,2)

MPEG

- **Frame 1 (I-frame)**

$$F1 = \begin{bmatrix} 10 & 10 & 20 & 20 \\ 10 & 10 & 20 & 20 \\ 30 & 30 & 40 & 40 \\ 30 & 30 & 40 & 40 \end{bmatrix}$$

Frame 2 (P-frame)

The objects shifted right by 1 pixel:

$$F2 = \begin{bmatrix} 0 & 10 & 10 & 20 \\ 0 & 10 & 10 & 20 \\ 0 & 30 & 30 & 40 \\ 0 & 30 & 30 & 40 \end{bmatrix}$$

Find Motion Vectors for Each Block

For block 1 at (0,0) in $F2$, search near (0,0) in $F1$:

Compare with block at (0,1) in $F1$: $\begin{bmatrix} 10 & 20 \\ 10 & 20 \end{bmatrix}$

Difference from $F2$ block:

$$\begin{bmatrix} 0 & 10 \\ 0 & 10 \end{bmatrix} - \begin{bmatrix} 10 & 20 \\ 10 & 20 \end{bmatrix} = \begin{bmatrix} -10 & -10 \\ -10 & -10 \end{bmatrix}$$

Error sum: 40

MPEG

- **Frame 1 (I-frame)**

$$F1 = \begin{bmatrix} 10 & 10 & 20 & 20 \\ 10 & 10 & 20 & 20 \\ 30 & 30 & 40 & 40 \\ 30 & 30 & 40 & 40 \end{bmatrix}$$

Frame 2 (P-frame)

The objects shifted right by 1 pixel:

$$F2 = \begin{bmatrix} 0 & 10 & 10 & 20 \\ 0 & 10 & 10 & 20 \\ 0 & 30 & 30 & 40 \\ 0 & 30 & 30 & 40 \end{bmatrix}$$

Find Motion Vectors for Each Block

For block 1 at (0,0) in $F2$, search near (0,0) in $F1$:

Compare with block at (0,0) in $F1$:

$$\begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}$$

Difference from $F2$ block:

$$\begin{bmatrix} 0 & 10 \\ 0 & 10 \end{bmatrix} - \begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix} = \begin{bmatrix} -10 & 0 \\ -10 & 0 \end{bmatrix}$$

Motion vector for block 1:
$\vec{v}_1 = (0,0)$ (no displacement).

Error sum: 20 (Better match)

MPEG

- **Frame 1 (I-frame)**

$$F1 = \begin{bmatrix} 10 & 10 & 20 & 20 \\ 10 & 10 & 20 & 20 \\ 30 & 30 & 40 & 40 \\ 30 & 30 & 40 & 40 \end{bmatrix}$$

Frame 2 (P-frame)

The objects shifted right by 1 pixel:

$$F2 = \begin{bmatrix} 0 & 10 & 10 & 20 \\ 0 & 10 & 10 & 20 \\ 0 & 30 & 30 & 40 \\ 0 & 30 & 30 & 40 \end{bmatrix}$$

Find Motion Vectors for Each Block

For block 2 at (0,2) in $F2$, search near (0,1) in $F1$:

Compare with block at (0,1) in $F1$:

$$\begin{bmatrix} 10 & 20 \\ 10 & 20 \end{bmatrix}$$

Motion vector for block 2:
$\vec{v}_2 = (0, -1)$ (shifted by 1).

Difference from $F2$ block:

$$\begin{bmatrix} 10 & 20 \\ 10 & 20 \end{bmatrix} - \begin{bmatrix} 10 & 20 \\ 10 & 20 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Error sum: 0 (Perfect match)

MPEG

- **Frame 1 (I-frame)**

$$F1 = \begin{bmatrix} 10 & 10 & 20 & 20 \\ 10 & 10 & 20 & 20 \\ 30 & 30 & 40 & 40 \\ 30 & 30 & 40 & 40 \end{bmatrix}$$

Frame 2 (P-frame)

The objects shifted right by 1 pixel:

$$F2 = \begin{bmatrix} 0 & 10 & 10 & 20 \\ 0 & 10 & 10 & 20 \\ 0 & 30 & 30 & 40 \\ 0 & 30 & 30 & 40 \end{bmatrix}$$

Find Motion Vectors for Each Block

For block 3 at (2,0) in $F2$, search near (2,0) in $F1$:

Compare with block at (2,0) in $F1$:

$$\begin{bmatrix} 30 & 30 \\ 30 & 30 \end{bmatrix}$$

Difference from $F2$ block:

$$\begin{bmatrix} 0 & 30 \\ 0 & 30 \end{bmatrix} - \begin{bmatrix} 30 & 30 \\ 30 & 30 \end{bmatrix} = \begin{bmatrix} -30 & 0 \\ -30 & 0 \end{bmatrix}$$

Motion vector for block 2:
$\vec{v}_3 = (0,0)$ (no displacement).

Error sum: 60 (Better match)

MPEG

- **Frame 1 (I-frame)**

$$F1 = \begin{bmatrix} 10 & 10 & 20 & 20 \\ 10 & 10 & 20 & 20 \\ 30 & 30 & 40 & 40 \\ 30 & 30 & 40 & 40 \end{bmatrix}$$

Frame 2 (P-frame)

The objects shifted right by 1 pixel:

$$F2 = \begin{bmatrix} 0 & 10 & 10 & 20 \\ 0 & 10 & 10 & 20 \\ 0 & 30 & 30 & 40 \\ 0 & 30 & 30 & 40 \end{bmatrix}$$

Find Motion Vectors for Each Block

For block 3 at (2,2) in $F2$, search near (2,1) in $F1$:

Compare with block at (2,1) in $F1$:

$$\begin{bmatrix} 30 & 40 \\ 30 & 40 \end{bmatrix}$$

Difference from $F2$ block:

$$\begin{bmatrix} 30 & 40 \\ 30 & 40 \end{bmatrix} - \begin{bmatrix} 30 & 40 \\ 30 & 40 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Motion vector for block 2:
$\vec{v}_4 = (0, -1)$ (shifted by 1).

Error sum: 0 (Perfect match)

MPEG

- **Frame 1 (I-frame)**

$$F1 = \begin{bmatrix} 10 & 10 & 20 & 20 \\ 10 & 10 & 20 & 20 \\ 30 & 30 & 40 & 40 \\ 30 & 30 & 40 & 40 \end{bmatrix}$$

Frame 2 (P-frame)

The objects shifted right by 1 pixel:

$$F2 = \begin{bmatrix} 0 & 10 & 10 & 20 \\ 0 & 10 & 10 & 20 \\ 0 & 30 & 30 & 40 \\ 0 & 30 & 30 & 40 \end{bmatrix}$$

Step 3: Calculate Residuals

Residual = $F2\ block\ -$ $F1$ block shifted by motion vector.

For block 2:

$$Residual = \begin{bmatrix} 10 & 20 \\ 10 & 20 \end{bmatrix} - \begin{bmatrix} 10 & 20 \\ 10 & 20 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Zero residual, no data needed.

For block 1:

$$Residual = \begin{bmatrix} 0 & 20 \\ 0 & 20 \end{bmatrix} - \begin{bmatrix} 10 & 0 \\ 10 & 0 \end{bmatrix} = \begin{bmatrix} -10 & 0 \\ -10 & 0 \end{bmatrix}$$

Residual is encoded.

MPEG

Step 4: Encode Motion Vectors + Residuals

- Store motion vectors (small data).
- Store residual blocks (mostly small values).
- Much less data than full frame.

Summary:

Step	Description
I-frame	Fully encoded first frame
Motion estimation	Find block movements between frames
Motion vectors	Compact representation of block displacements
Residual calculation	Difference after motion compensation
P-frame encoding	Encode motion vectors + residuals for compression

H.263 Quantization Algorithm and Encoding using DCT

H.263 Basics

- Designed for low bit-rate video conferencing.
- Uses block-based motion compensation (16×16 macroblocks).
- Adds advanced features like half-pixel motion compensation, optional multiple reference frames, and better variable-length coding.
- Similar overall structure to H.261 but more efficient.

Frame 1 (reference frame)

$$F1 = \begin{bmatrix} 100 & 100 & 120 & 120 \\ 100 & 100 & 120 & 120 \\ 140 & 140 & 160 & 160 \\ 140 & 140 & 160 & 160 \end{bmatrix}$$

Frame 2 (current frame)

Shifted 1 pixel right and 1 pixel down:

$$F2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 100 & 100 & 120 \\ 0 & 100 & 100 & 120 \\ 0 & 140 & 140 & 160 \end{bmatrix}$$

H.263 Quantization Algorithm and Encoding using DCT

Frame 1 (reference frame)

$$F1 = \begin{bmatrix} 100 & 100 & 120 & 120 \\ 100 & 100 & 120 & 120 \\ 140 & 140 & 160 & 160 \\ 140 & 140 & 160 & 160 \end{bmatrix}$$

Frame 2 (current frame)

Shifted 1 pixel right and 1 pixel down:

$$F2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 100 & 100 & 120 \\ 0 & 100 & 100 & 120 \\ 0 & 140 & 140 & 160 \end{bmatrix}$$

Step 1: Motion Estimation (16×16 block reduced to 4×4 here)

- Search for matching blocks in $F1$ to predict $F2$.
- For block at (1,1) in $F2$, best match is block at (0,0) in $F1$.
- Motion vector $\vec{v} = (-1, -1)$ (shift left 1, up 1).

Step 2: Motion Compensation

- Predict $F2$ block using $F1$ shifted by $\vec{v}$.

H.263 Quantization Algorithm and Encoding using DCT

Frame 1 (reference frame)

$$F1 = \begin{bmatrix} 100 & 100 & 120 & 120 \\ 100 & 100 & 120 & 120 \\ 140 & 140 & 160 & 160 \\ 140 & 140 & 160 & 160 \end{bmatrix}$$

Frame 2 (current frame)

Shifted 1 pixel right and 1 pixel down:

$$F2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 100 & 100 & 120 \\ 0 & 100 & 100 & 120 \\ 0 & 140 & 140 & 160 \end{bmatrix}$$

Step 1: Motion Estimation (16×16 block reduced to 4×4 here)

- Search for matching blocks in $F1$ to predict $F2$.
- For block at (1,1) in $F2$, best match is block at (0,0) in $F1$.
- Motion vector $\vec{v} = (-1, -1)$ (shift left 1, up 1).

Step 2: Motion Compensation

- Predict $F2$ block using $F1$ shifted by $\vec{v}$.

H.263 Quantization Algorithm and Encoding using DCT

Frame 1 (reference frame)

$$F1 = \begin{bmatrix} 100 & 100 & 120 & 120 \\ 100 & 100 & 120 & 120 \\ 140 & 140 & 160 & 160 \\ 140 & 140 & 160 & 160 \end{bmatrix}$$

Frame 2 (current frame)

Shifted 1 pixel right and 1 pixel down:

$$F2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 100 & 100 & 120 \\ 0 & 100 & 100 & 120 \\ 0 & 140 & 140 & 160 \end{bmatrix}$$

Step 3: Residual Calculation

Subtract predicted block from actual block in $F2$:

$$R = F2 - Shift(F1, \vec{v})$$

Example pixel residuals:

- For pixel (1,1):

$$R(1,1) = 100 - 100 = 0$$

- For pixel (3,3):

$$R(3,3) = 160 - 160 = 0$$

- **Residuals are zero — perfect prediction!**

H.263 Quantization Algorithm and Encoding using DCT

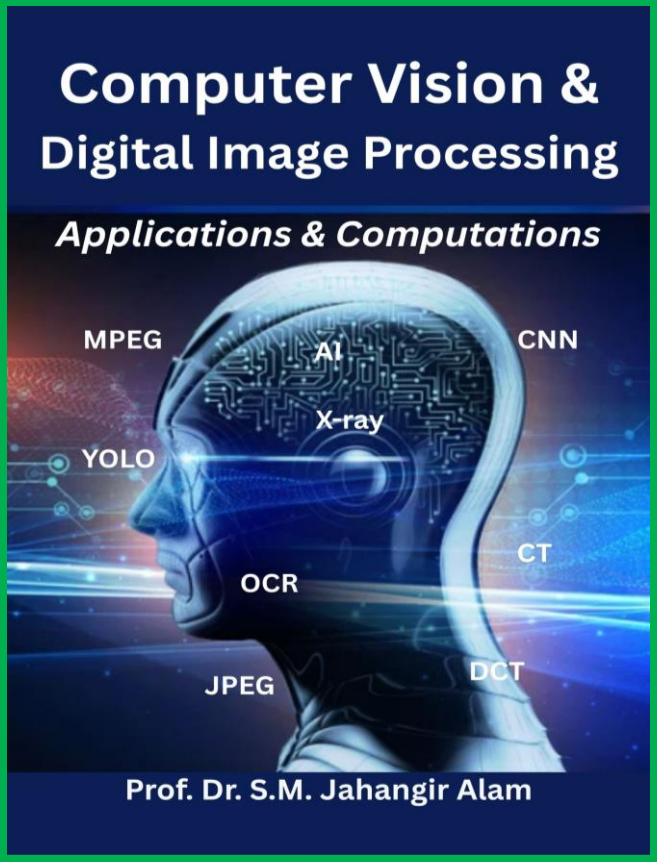
- **Step 4: DCT on Residuals**
- Since residuals are zero, DCT coefficients are zero → very efficient compression.
- **Step 5: Quantization and Encoding**
- Zero coefficients after quantization → encode efficiently.
- Encode motion vector $\vec{v} = (-1, -1)$ with efficient variable length codes.

Summary:

Step	Description
Motion Estimation	Find $\vec{v} = (-1, -1)$ for blocks
Motion Compensation	Predict current block from previous frame
Residual Calculation	Mostly zero residuals here
DCT and Quantization	Efficient encoding of residuals
Motion Vector Encoding	Compact representation of $\vec{v}$

Lecture 7.5

Image Compression



Instructor Name:

Prof. Dr. Jahangir Alam

Dept. of CSE, BAUST

Entropy-Based Schemes Using Huffman Coding

##A simple Entropy-Based Image Compression Scheme using Huffman coding, which is a classic entropy coding method.

- **Step 1: Given Image Pixel Values (small 1D example for simplicity)**
- Suppose we have a grayscale image segment with pixel values:
- Pixels=[100,100,101,102,100,101,102,102]
- **Step 2: Calculate the frequency of each pixel value**

Pixel Value	Count	Probability pipi
100	3	3/8=0.375
101	2	2/8=0.25
102	3	3/8=0.375

Entropy-Based Schemes Using Huffman Coding

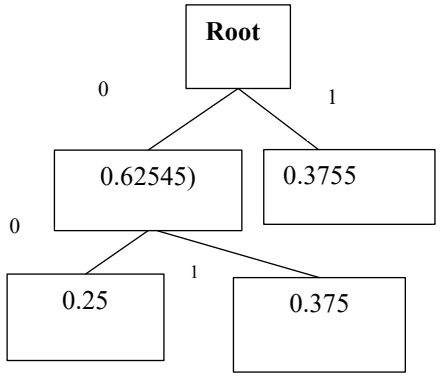
Pixel Value	Count	Probability pi
100	3	3/8=0.375
101	2	2/8=0.25
102	3	3/8=0.375

- **Step 3: Calculate Entropy *H***

$$H = -\sum_{i} P(i)\log_2 P(i)$$

- For 100: $-0.375 \times \log_2 0.375 \approx -0.375 \times (-1.415) = 0.531$
- For 101: $-0.25 \times \log_2 0.25 = -0.25 \times (-2) = 0.5$
- For 102: same as 100 = 0.531

$$H \approx 0.531 + 0.5 + 0.531 = 1.562\ bits/pixel$$



Entropy-Based Schemes Using Huffman Coding

Pixel Value	Count	Probability pi
100	3	3/8=0.375
101	2	2/8=0.25
102	3	3/8=0.375

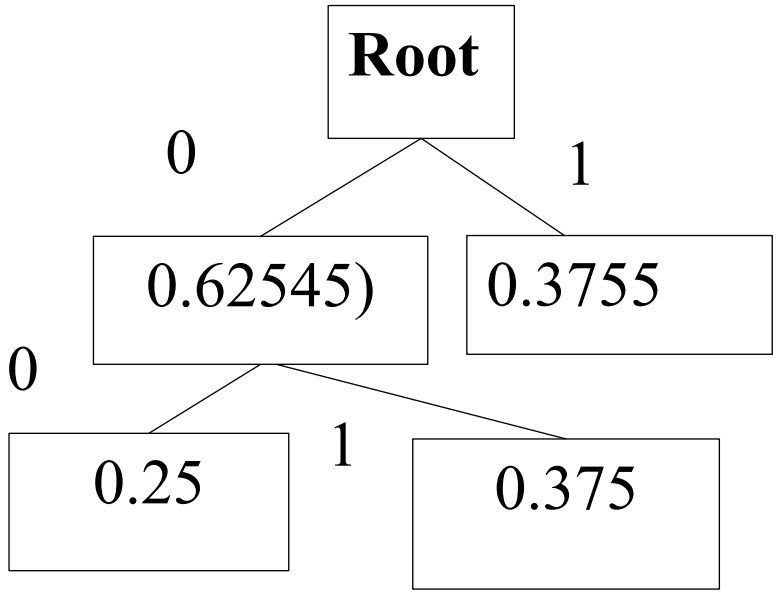
Combine two smallest: 101 (0.25) + 100 (0.375) = 0.625

Node	Probability
102	0.375
101+100	0.625

Step 5: Assign Codes

Combine these to root node 1.0.

- Root splits into:
  - 101+100 (0.625) → assign code 0
  - 102 (0.375) → assign code 1
- Split 101+100:
  - 101 (0.25) → assign code 00
  - 100 (0.375) → assign code 01



Step 6: Final Huffman Codes

Symbol	100	101	102
Code	01	00	1

Entropy-Based Schemes Using Huffman Coding

Step 7: Encode Original Pixel Sequence

[100,100,101,102,100,101,102,102]

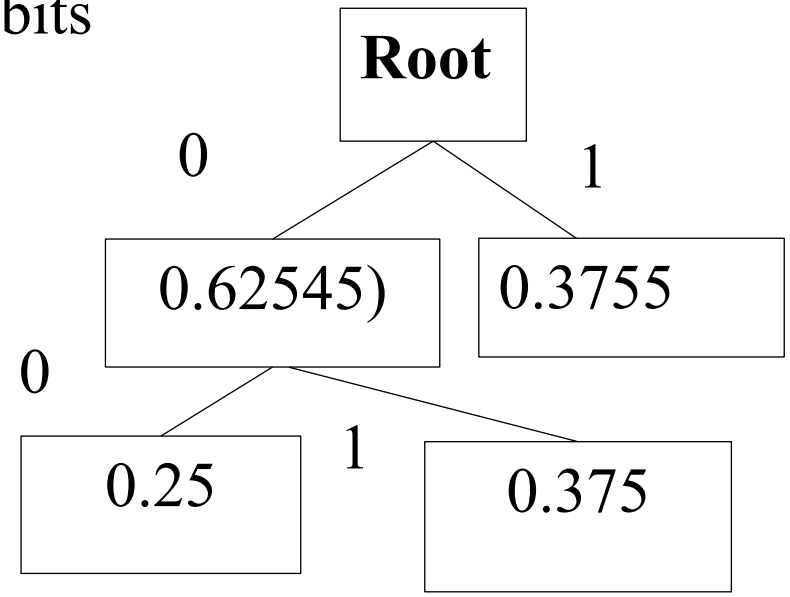
→ Encoded as:

01 01 00 1 01 00 1 1

Number of bits:

- 100 appears 3 times → 3 × 2 bits = 6 bits
- 101 appears 2 times → 2 × 2 bits = 4 bits
- 102 appears 3 times → 3 × 1 bit = 3 bits

Total bits = 6 + 4 + 3 = 13 bits



Entropy-Based Schemes Using Huffman Coding

Step 8: Compare with fixed-length coding

- Original pixel values can be represented with 8 bits each → $8 \times 8 = 64$ bits
- Compressed length = 13 bits → compression ratio $\approx 64/13 \approx 4.92$

Summary:

Step	Explanation
Frequency count	Compute symbol probabilities
Calculate entropy	The theoretical lower bound of coding length
Build Huffman tree	Create optimal prefix codes
Encode sequence	Use variable length codes
Compression ratio	Compare to fixed-length coding

Vector Quantization (VQ) for Image Pro.

- VQ is a lossy image compression technique that encodes blocks (vectors) of pixels instead of single pixel values.
- Based on the idea of replacing similar image blocks by the same “codeword” from a codebook — very much like how we reuse similar words in a language dictionary.
- VQ compresses an image by dividing it into blocks (vectors).
- Each block is approximated by the closest code vector from a codebook.
- Only the index of the closest code vector is stored/transmitted, reducing data size.

Vector Quantization (VQ) for Image Pro.

Step 1: Image divided into 2×2 blocks (vectors)

Suppose a small grayscale 4×4 image:

	1	2	3	4
1	52	55	61	59
2	62	59	55	90
3	63	65	66	113
4	58	70	70	120

Divided into four 2×2 blocks (vectors):

- Block1 (top-left): [52,55;62,59] → vector v1=[52,55,62,59]
- Block2 (top-right): [61,59;55,90] → v2=[61,59,55,90]
- Block3 (bottom-left): [63,65;58,70] → v3=[63,65,58,70]
- Block4 (bottom-right): [66,113;70,120] → v4=[66,113,70,120]

Step 2: Codebook with 2 code vectors (predefined or trained)

Suppose codebook vectors:

- c1=[55,55,60,60]
- c2=[65,110,70,115]

Vector Quantization (VQ) for Image Pro.

Step 3: Find closest code vector for each image block (using Euclidean distance)

Calculate distance, $d(v,c) = \sqrt{\sum(v_i - c_i)^2}$

Block 1 vs. c1:

$$d = \sqrt{(52-55)^2+(55-55)^2+(62-60)^2+(59-60)^2}$$
$$= \sqrt{14} \approx 3.74$$

Block 1 vs. c2:

$$d = \sqrt{(52-65)^2+(55-110)^2+(62-70)^2+(59-115)^2}$$
$$= \sqrt{6394} \approx 79.94$$

→ Closest: c1

Vector Quantization (VQ) for Image Pro.

Block 2 vs. c1:

$$d = \sqrt{(61-55)^2+(59-55)^2+(55-60)^2+(90-60)^2} = \sqrt{977} \approx 31.27$$

Block 2 vs. c2:

$$d = \sqrt{(61-65)^2+(59-110)^2+(55-70)^2+(90-115)^2} = \sqrt{3467} \approx 58.87$$

→ Closest: c1

Block 3 vs. c1:

$$d = \sqrt{(63-55)^2+(65-55)^2+(58-60)^2+(70-60)^2} = \sqrt{268} \approx 16.37$$

Block 3 vs. c2:

$$d = \sqrt{(63-65)^2+(65-110)^2+(58-70)^2+(70-115)^2} = \sqrt{4198} \approx 64.78$$

→ Closest: c1

Vector Quantization (VQ) for Image Pro.

Block 4 vs. c1:

$$d = \sqrt{(66-55)^2+(113-55)^2+(70-60)^2+(120-60)^2}$$
$$= \sqrt{7185} \approx 84.73$$

Block 4 vs. c2:

$$d = \sqrt{(66-65)^2+(113-110)^2+(70-70)^2+(120-115)^2}$$
$$= \sqrt{35} \approx 5.92$$

→ Closest: c2

Block	Closest Code Vector	Codebook index
1	c1	1
2	c1	1
3	c1	1
4	c2	2

Vector Quantization (VQ) for Image Pro.

Step 5: Compressed representation

Instead of sending all pixels, send:

- The codebook (c1, c2) vectors
- The indices: [1, 1, 1, 2]

Summary:

- Vector quantization approximates blocks by nearest codebook vectors.
- Compression achieved by storing indices instead of raw blocks.
- Reconstruction uses code vectors corresponding to indices.

Video Compression Coding

Frame 1 (I-frame, intra-coded, coded independently):

10	12	14	16
10	12	14	16
10	12	14	16
10	12	14	16

Frame 2 (P-frame, predicted from Frame 1):

11	13	15	17
11	13	15	17
11	13	15	17
11	13	15	17

Frame 3 (P-frame, predicted from Frame 2):

12	14	16	18
12	14	16	18
12	14	16	18
12	14	16	18

Video Compression Coding

Step 1: Encode Frame 1 (I-frame)
Since Frame 1 is an I-frame, encode all pixel values as-is intra-coded.

11	13	15	17
11	13	15	17
11	13	15	17
11	13	15	17

10	12	14	16
10	12	14	16
10	12	14	16
10	12	14	16

Step 2: Encode Frame 2 (P-frame) via Motion Estimation and Residual

- Since Frame 2 is similar to Frame 1 but each pixel increased by 1, motion estimation finds zero motion vectors (no pixel shift).
- Residual frame = Frame 2 - Frame 1

Calculate residual:

11-10=1	13-12=1	15-14=1	17-16=1
1	1	1	1
1	1	1	1
1	1	1	1

Residual block is uniform 1's. Only residuals and motion vectors (which are zero here) need to be encoded, which is much smaller than full frame.

Video Compression Coding

12	14	16	18
12	14	16	18
12	14	16	18
12	14	16	18

11	13	15	17
11	13	15	17
11	13	15	17
11	13	15	17

Step 3: Encode Frame 3 (P-frame) similarly

Residual = Frame 3 - Frame 2

12-11=1	14-13=1	16-15=1	18-17=1
1	1	1	1
1	1	1	1
1	1	1	1

Again, residuals of 1, motion vectors zero.

Video Compression Coding

Step 4: Compression coding

- Instead of encoding full frames 2 and 3, encode only:
  - Motion vectors (zero vectors, very small info)
  - Residuals (all ones, easy to compress)

This reduces bitrate significantly.

Summary Table:

Frame	Coding Type	Data Encoded
1	I-frame	Full pixel data (all pixels)
2	P-frame	Motion vectors (zero) + residuals (all ones)
3	P-frame	Motion vectors (zero) + residuals (all ones)

Image Compression Standards, Formats, and Containers

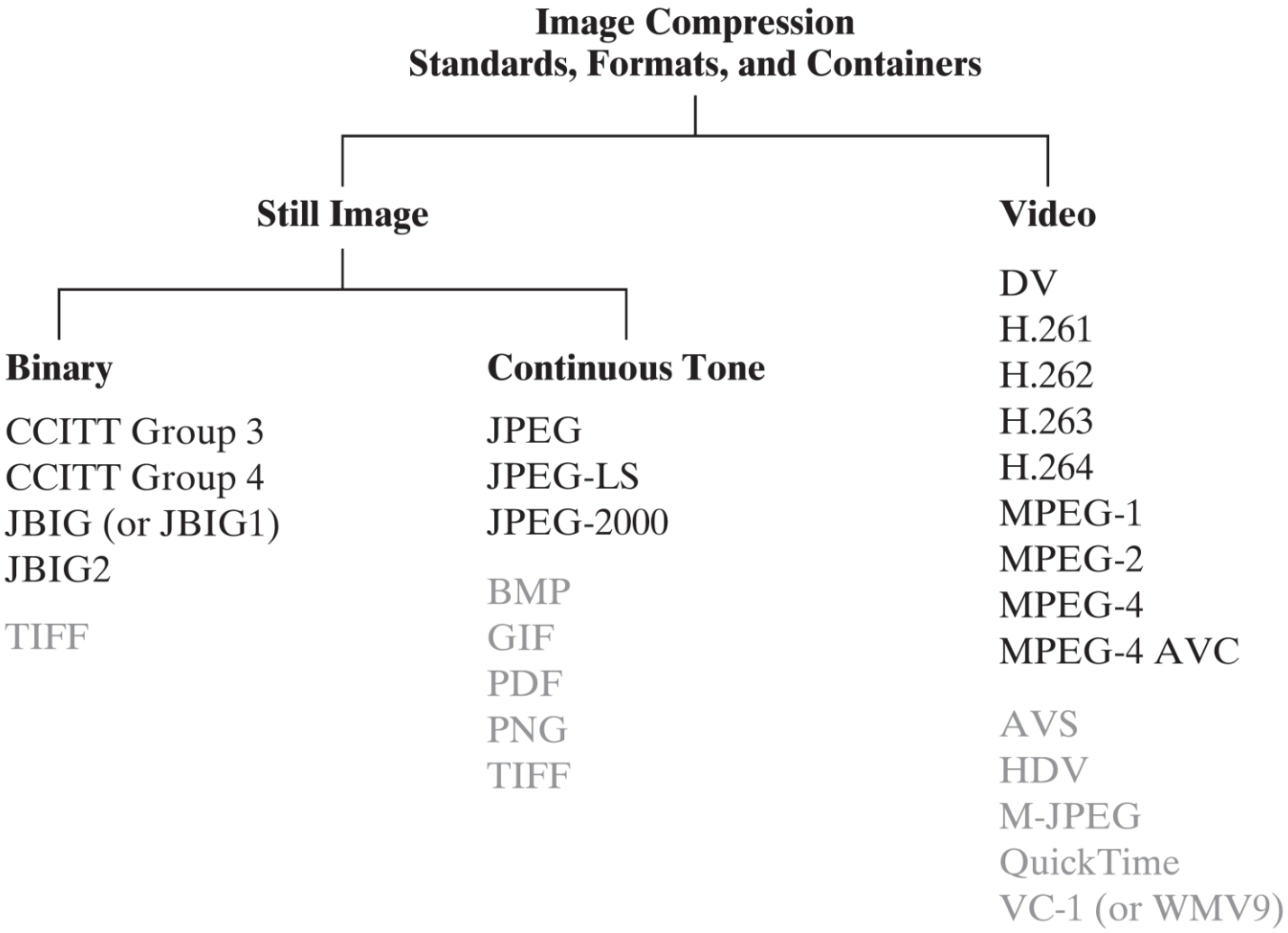


Image Compression Standards, Formats, and Containers

Name	Organization	Description
Bi-Level Still Images		
CCITT Group 3	ITU-T	Designed as a facsimile (FAX) method for transmitting binary documents over telephone lines. Supports 1-D and 2-D run-length [8.2.5] and Huffman [8.2.1] coding.
CCITT Group 4	ITU-T	A simplified and streamlined version of the CCITT Group 3 standard supporting 2-D run-length coding only.
JBIG *or* JBIG1	ISO/IEC/ ITU-T	A *Joint Bi-level Image Experts Group* standard for progressive, lossless compression of bi-level images. Continuous-tone images of up to 6 bits/pixel can be coded on a bit-plane basis [8.2.7]. Context sensitive arithmetic coding [8.2.3] is used and an initial low resolution version of the image can be gradually enhanced with additional compressed data.
JBIG2	ISO/IEC/ ITU-T	A follow-on to JBIG1 for bi-level images in desktop, Internet, and FAX applications. The compression method used is content based, with dictionary based methods [8.2.6] for text and halftone regions, and Huffman [8.2.1] or arithmetic coding [8.2.3] for other image content. It can be lossy or lossless.
Continuous-Tone Still Images		
JPEG	ISO/IEC/ ITU-T	A *Joint Photographic Experts Group* standard for images of photographic quality. Its lossy *baseline coding system* (most commonly implemented) uses quantized discrete cosine transforms (DCT) on 8×8 image blocks [8.2.8], Huffman [8.2.1], and run-length [8.2.5] coding. It is one of the most popular methods for compressing images on the Internet.
JPEG-LS	ISO/IEC/ ITU-T	A lossless to near-lossless standard for continuous tone images based on adaptive prediction [8.2.9], context modeling [8.2.3], and Golomb coding [8.2.2].
JPEG-2000	ISO/IEC/ ITU-T	A follow-on to JPEG for increased compression of photographic quality images. Arithmetic coding [8.2.3] and quantized discrete wavelet transforms (DWT) [8.2.10] are used. The compression can be lossy or lossless.

Name	Organization	Description
Video		
DV	IEC	*Digital Video*. A video standard tailored to home and semiprofessional video production applications and equipment—like electronic news gathering and camcorders. Frames are compressed independently for uncomplicated editing using a DCT-based approach [8.2.8] similar to JPEG.
H.261	ITU-T	A two-way videoconferencing standard for ISDN (*integrated services digital network*) lines. It supports non-interlaced 352×288 and 176×144 resolution images, called CIF (*Common Intermediate Format*) and QCIF (*Quarter CIF*), respectively. A DCT-based compression approach [8.2.8] similar to JPEG is used, with frame-to-frame prediction differencing [8.2.9] to reduce temporal redundancy. A block-based technique is used to compensate for motion between frames.
H.262	ITU-T	See MPEG-2 below.
H.263	ITU-T	An enhanced version of H.261 designed for ordinary telephone modems (i.e., 28.8 Kb/s) with additional resolutions: SQCIF (*Sub-Quarter* CIF 128×96), 4CIF (704×576), and 16CIF (1408×512).
H.264	ITU-T	An extension of H.261–H.263 for videoconferencing, Internet streaming, and television broadcasting. It supports prediction differences within frames [8.2.9], variable block size integer transforms (rather than the DCT), and context adaptive arithmetic coding [8.2.3].
MPEG-1	ISO/IEC	A *Motion Pictures Expert Group* standard for CD-ROM applications with non-interlaced video at up to 1.5 Mb/s. It is similar to H.261 but frame predictions can be based on the previous frame, next frame, or an interpolation of both. It is supported by almost all computers and DVD players.
MPEG-2	ISO/IEC	An extension of MPEG-1 designed for DVDs with transfer rates to 15 Mb/s. Supports interlaced video and HDTV. It is the most successful video standard to date.
MPEG-4	ISO/IEC	An extension of MPEG-2 that supports variable block sizes and prediction differencing [8.2.9] within frames.
MPEG-4 AVC	ISO/IEC	MPEG-4 Part 10 *Advanced Video Coding* (AVC). Identical to H.264 above.

Name	Organization	Description
Continuous-Tone Still Images		
BMP	Microsoft	*Windows Bitmap*. A file format used mainly for simple uncompressed images.
GIF	CompuServe	*Graphic Interchange Format*. A file format that uses lossless LZW coding [8.2.4] for 1- through 8-bit images. It is frequently used to make small animations and short low resolution films for the World Wide Web.
PDF	Adobe Systems	*Portable Document Format*. A format for representing 2-D documents in a device and resolution independent way. It can function as a container for JPEG, JPEG 2000, CCITT, and other compressed images. Some PDF versions have become ISO standards.
PNG	*World Wide Web Consortium* (W3C)	*Portable Network Graphics*. A file format that losslessly compresses full color images with transparency (up to 48 bits/pixel) by coding the difference between each pixel's value and a predicted value based on past pixels [8.2.9].
TIFF	Aldus	*Tagged Image File Format*. A flexible file format supporting a variety of image compression standards, including JPEG, JPEG-LS, JPEG-2000, JBIG2, and others.
Video		
AVS	MII	*Audio-Video Standard*. Similar to H.264 but uses exponential Golomb coding [8.2.2]. Developed in China.
HDV	Company consortium	*High Definition Video*. An extension of DV for HD television that uses MPEG-2 like compression, including temporal redundancy removal by prediction differencing [8.2.9].
M-JPEG	Various companies	*Motion JPEG*. A compression format in which each frame is compressed independently using JPEG.
Quick-Time	Apple Computer	A media container supporting DV, H.261, H.262, H.264, MPEG-1, MPEG-2, MPEG-4, and other video compression formats.
VC-1 WMV9	SMPTE Microsoft	The most used video format on the Internet. Adopted for HD and *Blu-ray* high-definition DVDs. It is similar to H.264/AVC, using an integer DCT with varying block sizes [8.2.8 and 8.2.9] and context dependent variable-length code tables [8.2.1]—but no predictions within frames.

Popular image compression standards

Name	Organization	Description
Continuous-Tone Still Images		
BMP	Microsoft	*Windows Bitmap*. A file format used mainly for simple uncompressed images.
GIF	CompuServe	*Graphic Interchange Format*. A file format that uses lossless LZW coding [8.2.4] for 1- through 8-bit images. It is frequently used to make small animations and short low resolution films for the World Wide Web.
PDF	Adobe Systems	*Portable Document Format*. A format for representing 2-D documents in a device and resolution independent way. It can function as a container for JPEG, JPEG 2000, CCITT, and other compressed images. Some PDF versions have become ISO standards.
PNG	*World Wide Web Consortium* (W3C)	*Portable Network Graphics*. A file format that losslessly compresses full color images with transparency (up to 48 bits/pixel) by coding the difference between each pixel's value and a predicted value based on past pixels [8.2.9].
TIFF	Aldus	*Tagged Image File Format*. A flexible file format supporting a variety of image compression standards, including JPEG, JPEG-LS, JPEG-2000, JBIG2, and others.

Name	Organization	Description
Continuous-Tone Still Images		
BMP	Microsoft	*Windows Bitmap*. A file format used mainly for simple uncompressed images.
GIF	CompuServe	*Graphic Interchange Format*. A file format that uses lossless LZW coding [8.2.4] for 1- through 8-bit images. It is frequently used to make small animations and short low resolution films for the World Wide Web.
PDF	Adobe Systems	*Portable Document Format*. A format for representing 2-D documents in a device and resolution independent way. It can function as a container for JPEG, JPEG 2000, CCITT, and other compressed images. Some PDF versions have become ISO standards.
PNG	*World Wide Web Consortium* (W3C)	*Portable Network Graphics*. A file format that losslessly compresses full color images with transparency (up to 48 bits/pixel) by coding the difference between each pixel's value and a predicted value based on past pixels [8.2.9].
TIFF	Aldus	*Tagged Image File Format*. A flexible file format supporting a variety of image compression standards, including JPEG, JPEG-LS, JPEG-2000, JBIG2, and others.
Video		
AVS	MII	*Audio-Video Standard*. Similar to H.264 but uses exponential Golomb coding [8.2.2]. Developed in China.
HDV	Company consortium	*High Definition Video*. An extension of DV for HD television that uses MPEG-2 like compression, including temporal redundancy removal by prediction differencing [8.2.9].
M-JPEG	Various companies	*Motion JPEG*. A compression format in which each frame is compressed independently using JPEG.
Quick-Time	Apple Computer	A media container supporting DV, H.261, H.262, H.264, MPEG-1, MPEG-2, MPEG-4, and other video compression formats.
VC-1 WMV9	SMPTE Microsoft	The most used video format on the Internet. Adopted for HD and *Blu-ray* high-definition DVDs. It is similar to H.264/AVC, using an integer DCT with varying block sizes [8.2.8 and 8.2.9] and context dependent variable-length code tables [8.2.1]—but no predictions within frames.

Thank You